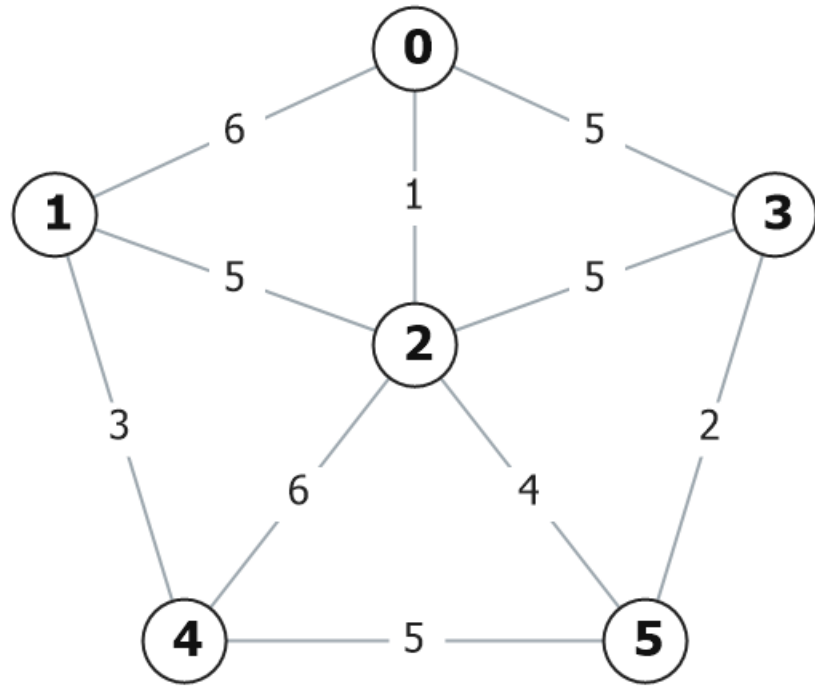




กราฟที่ 1 | ข้อมูลและข้อตกลง

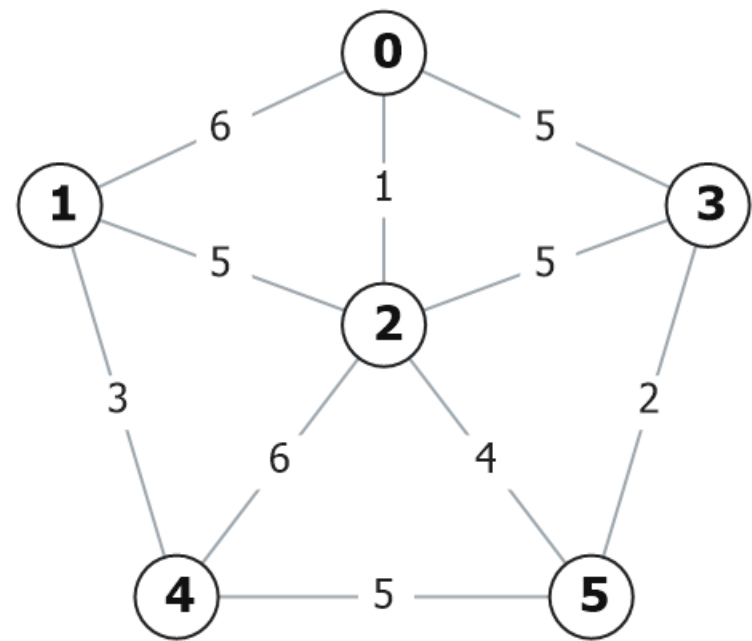
กราฟไม่มีทิศทางและมีน้ำหนัก | เริ่มจากจุด 1 ตามโจทย์



1. Adjacency Matrix ใช้น้ำหนักแทนค่า 1
2. Adjacency List แยกจุด / น้ำหนัก / ตัวชี้
3. DFS ใช้สแตก เลือกเพื่อนบ้านเลขน้อยก่อน
4. BFS ใช้คิว เลือกเพื่อนบ้านเลขน้อยก่อน
5. MST ใช้ Prim เลือกเส้นน้ำหนักน้อยสุด
6. Shortest Path ใช้ Dijkstra เลือกน้ำหนักสะสม

กราฟที่ 1 | 1. Adjacency Matrix

กราฟมีน้ำหนัก: ใส่ น้ำหนักเมื่อมีเส้นเชื่อม และใส่ 0 เมื่อไม่มีเส้นเชื่อม

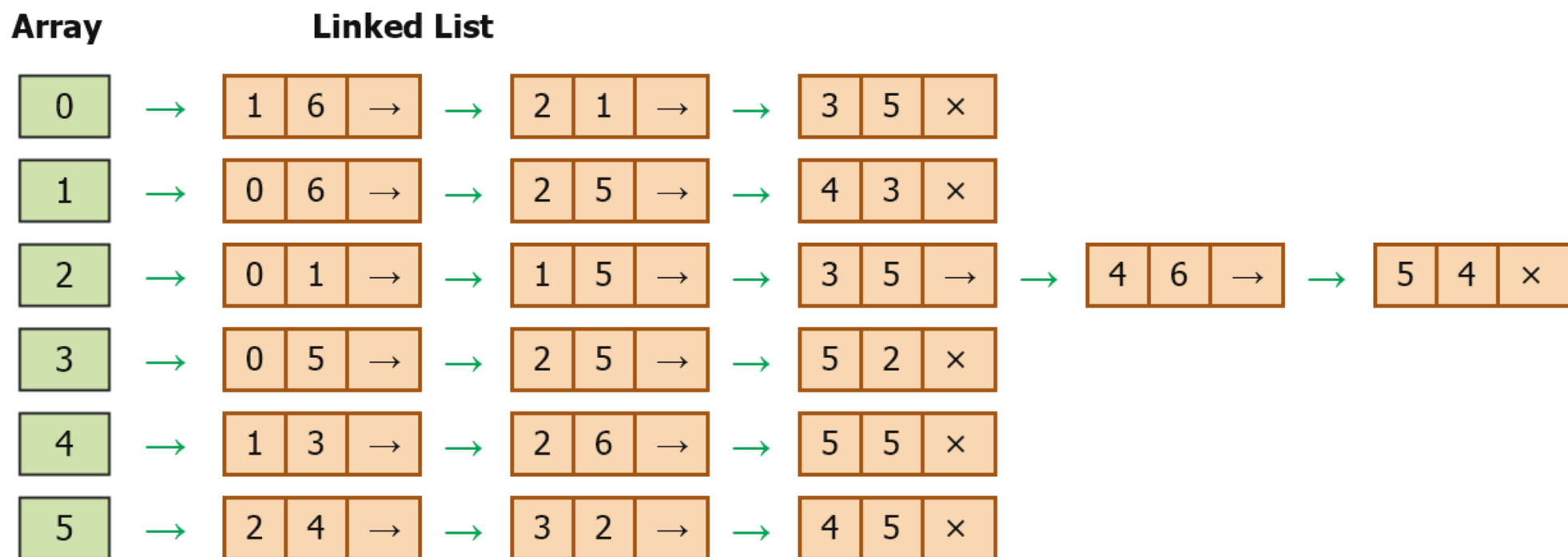


From / To	0	1	2	3	4	5
0	0	6	1	5	0	0
1	6	0	5	0	3	0
2	1	5	0	5	6	4
3	5	0	5	0	0	2
4	0	3	6	0	0	5
5	0	0	4	2	5	0

เมทริกซ์สมมาตร เพราะเส้นเชื่อมไม่มีทิศทาง

● กราฟที่ 1 | 2. Adjacency List

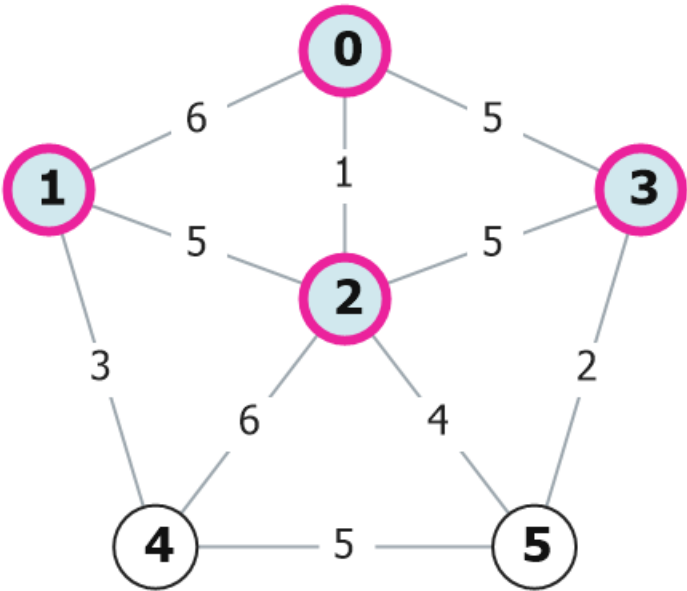
ช่องต้นทาง (Array) ชี้ไปยังลิงก์ลิสต์ | แต่ละโหนดมี [จุดข้างเคียง | น้ำหนัก | ตัวชี้]



× = NULL | มี 10 เส้น จึงมีโหนดในลิสต์ทั้งหมด 20 โหนด

กราฟที่ 1 | Depth-first traversal: Push

เลือกเพื่อนบ้านเลขน้อยก่อน | เยี่ยมชมแล้ว Push | ไม่มีเพื่อนบ้านใหม่ให้ Pop จนสแตกว่าง



Push 1

Path:
1

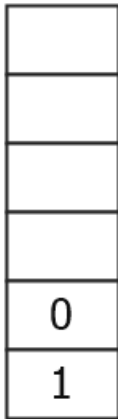
สแตกล้างทำงาน
Top ↑



Push 0

Path:
1 → 0

สแตกล้างทำงาน
Top ↑



Push 2

Path:
1 → 0 → 2

สแตกล้างทำงาน
Top ↑



Push 3

Path:
1 → 0 → 2 → 3

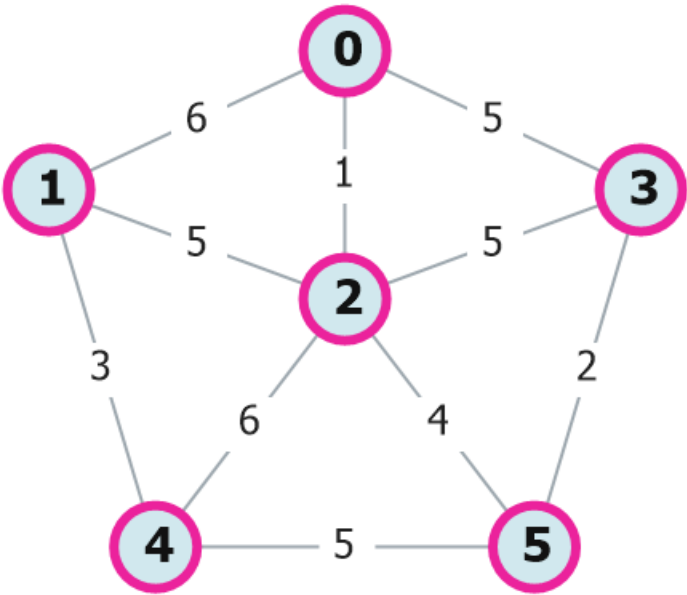
สแตกล้างทำงาน
Top ↑



Path = ลำดับเยี่ยมชม ไม่ใช่การนำจุดที่ย้อนกลับมาเขียนซ้ำ

กราฟที่ 1 | Depth-first traversal: Push

เลือกเพื่อนบ้านเลขน้อยก่อน | เยี่ยมชมแล้ว Push | ไม่มีเพื่อนบ้านใหม่ให้ Pop จนสแตกว่าง



Push 5

Path:
 $1 \rightarrow 0 \rightarrow 2 \rightarrow 3$
5

สแตกหลังทำงาน
Top ↑

5
3
2
0
1

Push 4

Path:
 $1 \rightarrow 0 \rightarrow 2 \rightarrow 3$
 $5 \rightarrow 4$

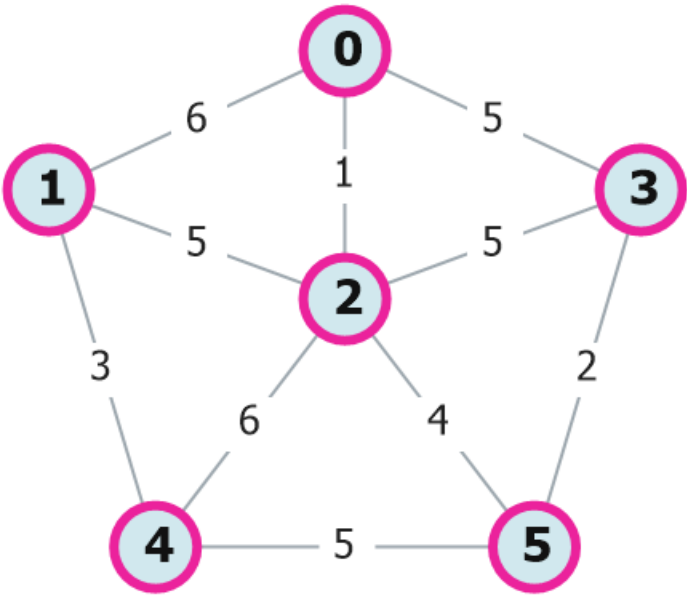
สแตกหลังทำงาน
Top ↑

4
5
3
2
0
1

Path = ลำดับเยี่ยมชม ไม่ใช่การนำจุดที่ย้อนกลับมาเขียนซ้ำ

กราฟที่ 1 | Depth-first traversal: Pop และย้อนกลับ

เลือกเพื่อนบ้านเลขน้อยก่อน | เยี่ยมชมแล้ว Push | ไม่มีเพื่อนบ้านใหม่ให้ Pop จนสแตกว่าง



Pop 4

Path:
 $1 \rightarrow 0 \rightarrow 2 \rightarrow 3$
 $5 \rightarrow 4$

สแตกล้างทำงาน
Top ↑

5
3
2
0
1

Pop 5

Path:
 $1 \rightarrow 0 \rightarrow 2 \rightarrow 3$
 $5 \rightarrow 4$

สแตกล้างทำงาน
Top ↑

3
2
0
1

Pop 3

Path:
 $1 \rightarrow 0 \rightarrow 2 \rightarrow 3$
 $5 \rightarrow 4$

สแตกล้างทำงาน
Top ↑

2
0
1

Pop 2

Path:
 $1 \rightarrow 0 \rightarrow 2 \rightarrow 3$
 $5 \rightarrow 4$

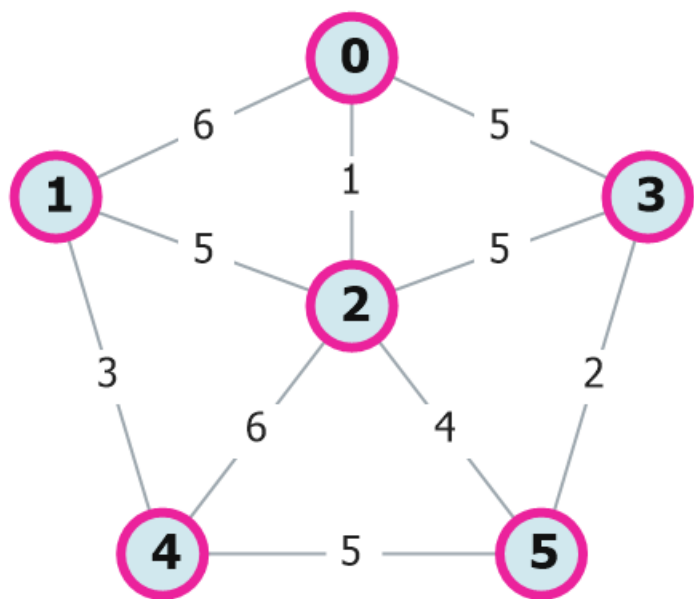
สแตกล้างทำงาน
Top ↑

0
1

Path = ลำดับเยี่ยมชม ไม่ใช่การนำจุดที่ย้อนกลับมาเขียนซ้ำ

กราฟที่ 1 | Depth-first traversal: Pop และย้อนกลับ

เลือกเพื่อนบ้านเลขน้อยก่อน | เยี่ยมชมแล้ว Push | ไม่มีเพื่อนบ้านใหม่ให้ Pop จนสแตกว่าง



Pop 0

Path:
 $1 \rightarrow 0 \rightarrow 2 \rightarrow 3$
 $5 \rightarrow 4$

สแตกหลังทำงาน

Top ↑



Pop 1

Path:
 $1 \rightarrow 0 \rightarrow 2 \rightarrow 3$
 $5 \rightarrow 4$

สแตกหลังทำงาน

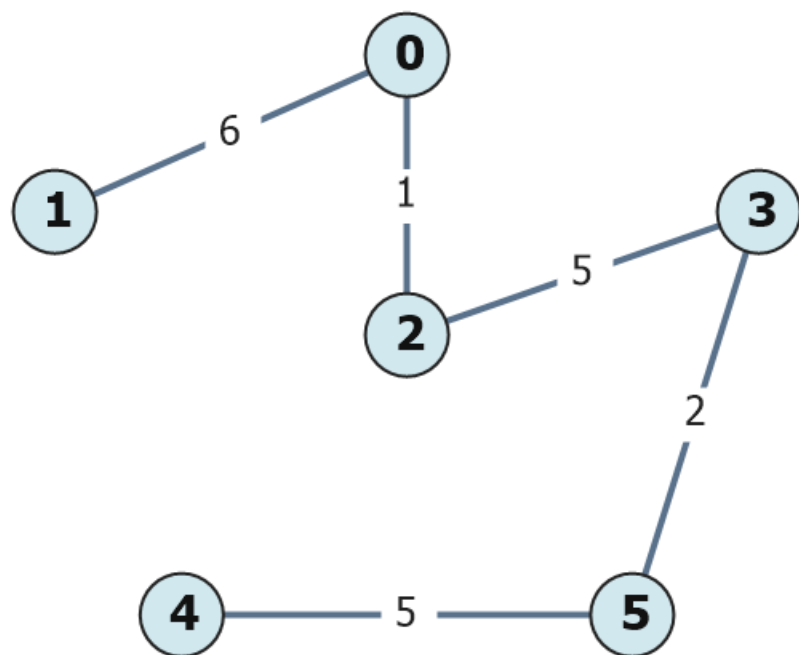
Top ↑



Path = ลำดับเยี่ยมชม ไม่ใช่การนำจุดที่ย้อนกลับมาเขียนซ้ำ

● กราฟที่ 1 | DFS: Path ที่ได้

หลัง Pop จุดเริ่มต้น สแตกว่าง จึงหยุดการทำงาน



Path:

$1 \rightarrow 0 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4$

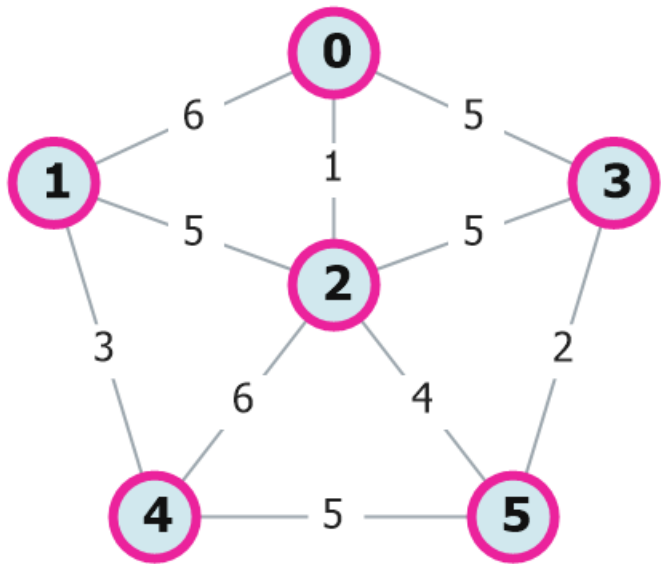
ต้นไม้ DFS:

1-0, 0-2, 2-3, 3-5

5-4

กราฟที่ 1 | 4. Breadth-first traversal

เริ่มคิว [1] | ค้นพบแล้วทำเครื่องหมายทันที | เต็มท้ายคิวและนำออกจากหัวคิว



R1: นำ 1 ออกจากคิว

PATH: 1

0	2	4		
---	---	---	--	--

R2: นำ 0 ออกจากคิว

PATH: 1-0

2	4	3		
---	---	---	--	--

R3: นำ 2 ออกจากคิว

PATH: 1-0-2

4	3	5		
---	---	---	--	--

R4: นำ 4 ออกจากคิว

PATH: 1-0-2-4

3	5			
---	---	--	--	--

R5: นำ 3 ออกจากคิว

PATH: 1-0-2-4-3

5				
---	--	--	--	--

R6: นำ 5 ออกจากคิว

PATH: 1-0-2-4-3-5

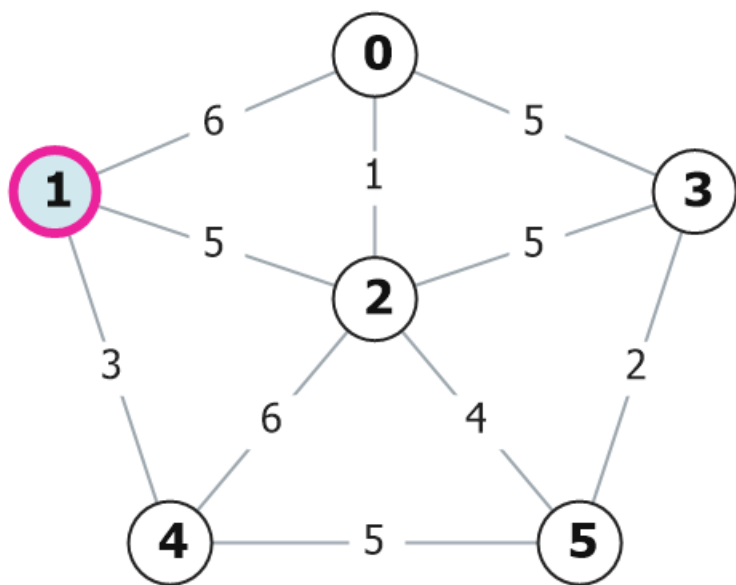
--	--	--	--	--

หัวคิวอยู่ซ้าย | คิวว่างในรอบสุดท้าย จบการทำงาน

กราฟที่ 1 | 5. Minimum Spanning Tree (Prim)

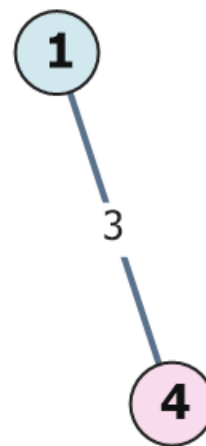
รอบที่ 1 | พิจารณาเส้นจากจุดใน Tree ไปยังจุดที่ยังไม่อยู่ใน Tree

กราฟเดิม: พิจารณาจุด 1



เส้นที่เลือกได้ (เรียงตามน้ำหนัก):
1-4(3), 1-2(5), 0-1(6)

เลือกเส้น 1-4 น้ำหนัก 3 เพิ่มใน Tree

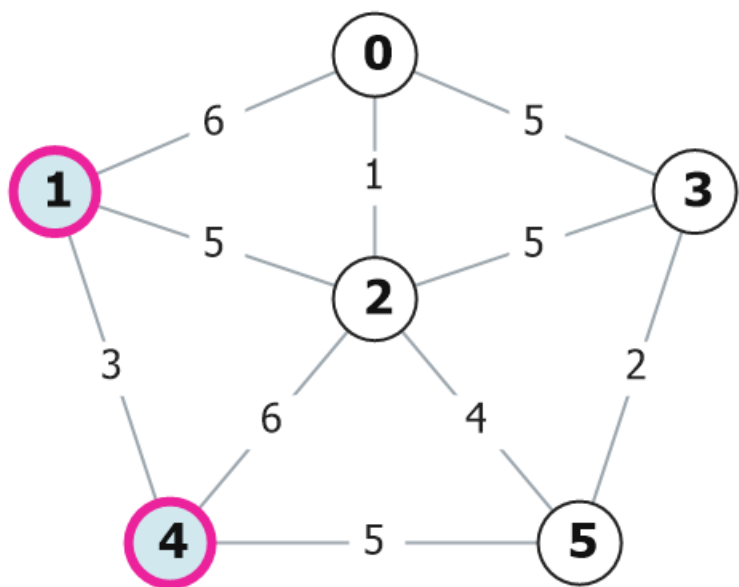


น้ำหนักรวมสะสม = 3

กราฟที่ 1 | 5. Minimum Spanning Tree (Prim)

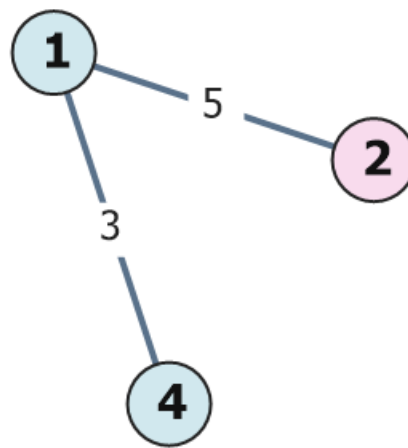
รอบที่ 2 | พิจารณาเส้นจากจุดใน Tree ไปยังจุดที่ยังไม่อยู่ใน Tree

กราฟเดิม: พิจารณาจุด 1, 4



เส้นที่เลือกได้ (เรียงตามน้ำหนัก):
1-2(5), 4-5(5), 0-1(6), 2-4(6)

เลือกเส้น 1-2 น้ำหนัก 5 เพิ่มใน Tree

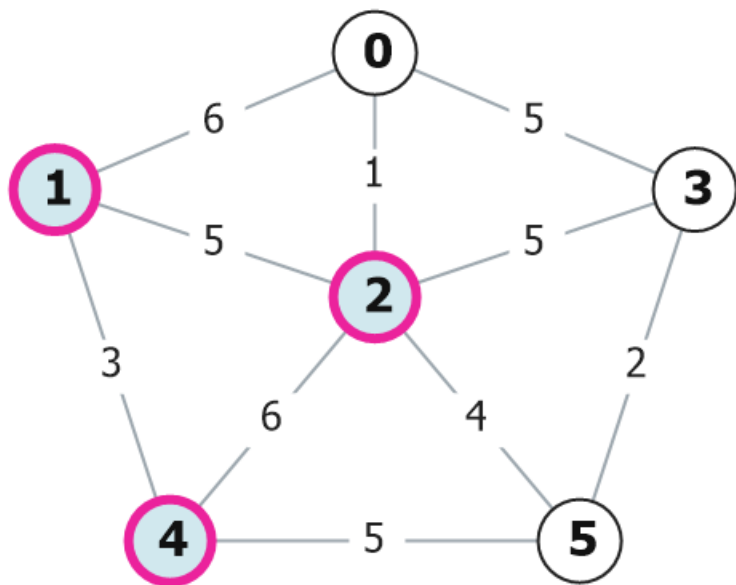


น้ำหนักรวมสะสม = 8

กราฟที่ 1 | 5. Minimum Spanning Tree (Prim)

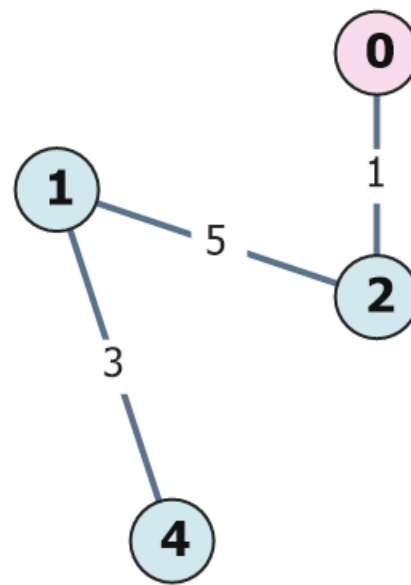
รอบที่ 3 | พิจารณาเส้นจากจุดใน Tree ไปยังจุดที่ยังไม่อยู่ใน Tree

กราฟเดิม: พิจารณาจุด 1, 2, 4



เส้นที่เลือกได้ (เรียงตามน้ำหนัก):
0-2(1), 2-5(4), 2-3(5), 4-5(5), 0-1(6)

เลือกเส้น 2-0 น้ำหนัก 1 เพิ่มใน Tree

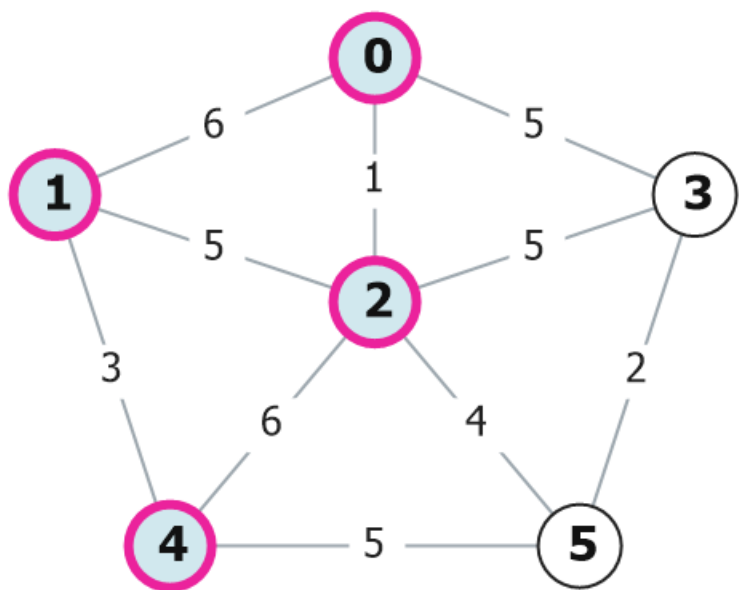


น้ำหนักรวมสะสม = 9

กราฟที่ 1 | 5. Minimum Spanning Tree (Prim)

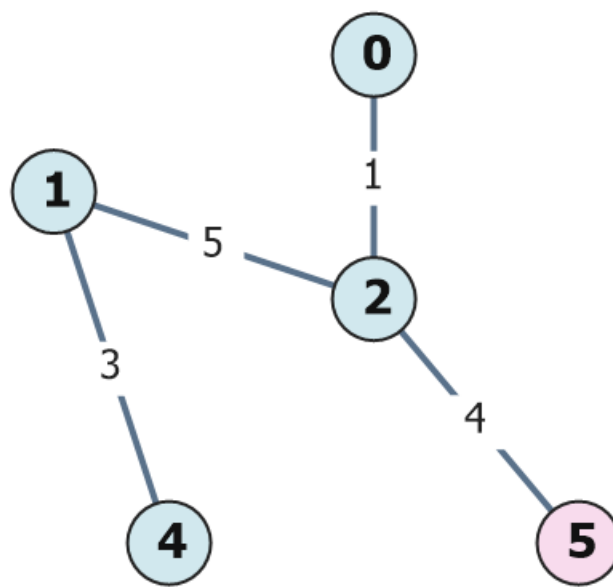
รอบที่ 4 | พิจารณาเส้นจากจุดใน Tree ไปยังจุดที่ยังไม่อยู่ใน Tree

กราฟเดิม: พิจารณาจุด 0, 1, 2, 4



เส้นที่เลือกได้ (เรียงตามน้ำหนัก):
2-5(4), 0-3(5), 2-3(5), 4-5(5)

เลือกเส้น 2-5 น้ำหนัก 4 เพิ่มใน Tree

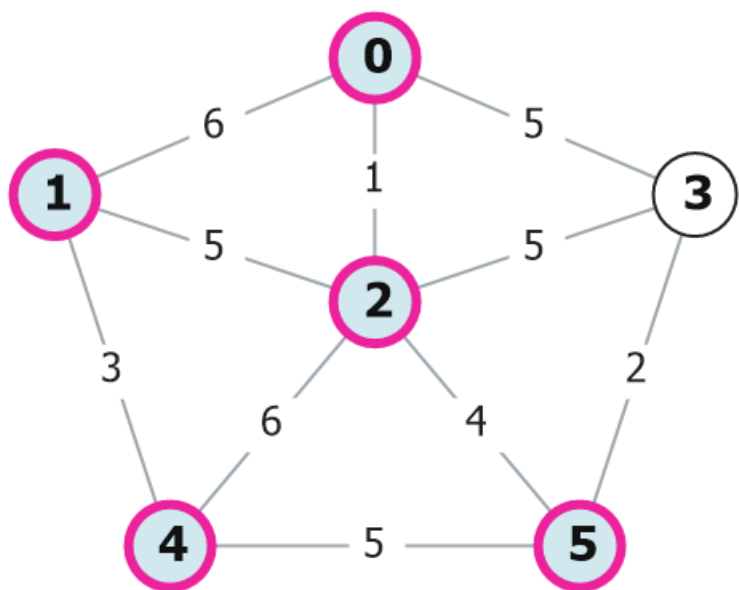


น้ำหนักรวมสะสม = 13

กราฟที่ 1 | 5. Minimum Spanning Tree (Prim)

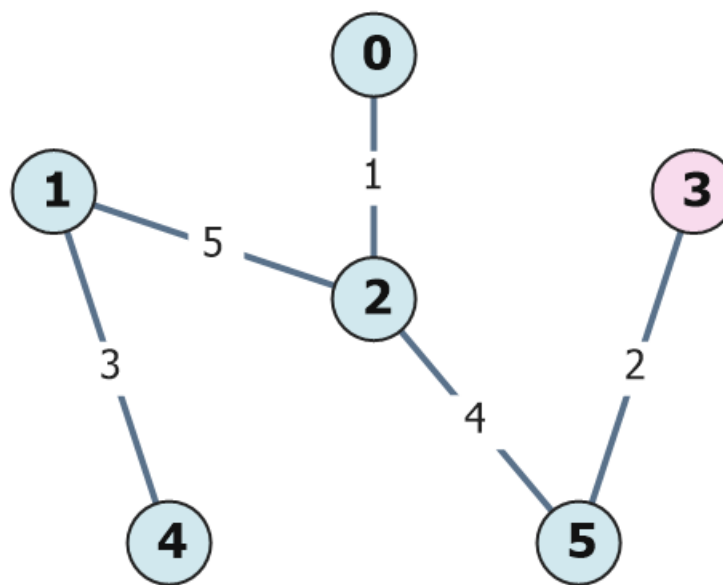
รอบที่ 5 | พิจารณาเส้นจากจุดใน Tree ไปยังจุดที่ยังไม่อยู่ใน Tree

กราฟเดิม: พิจารณาจุด 0, 1, 2, 4, 5



เส้นที่เลือกได้ (เรียงตามน้ำหนัก):
3-5(2), 0-3(5), 2-3(5)

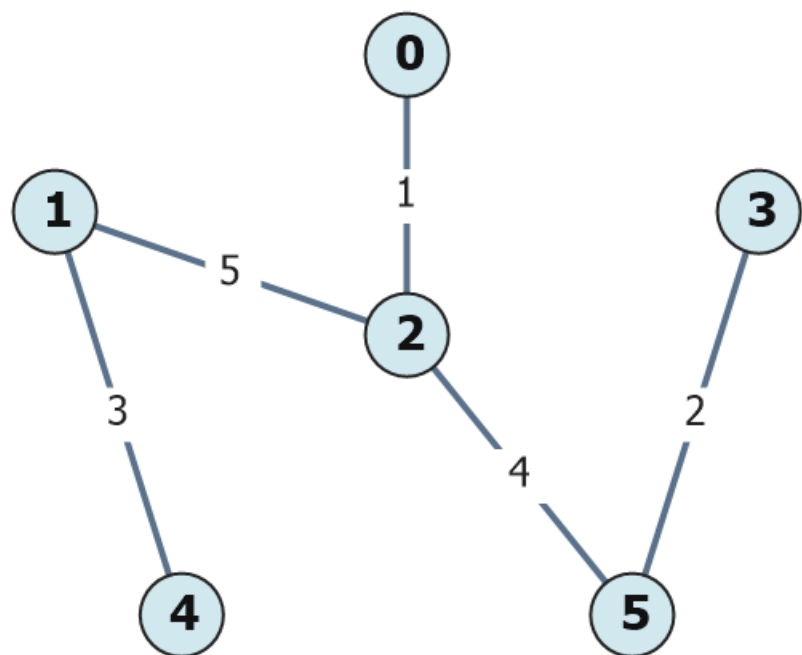
เลือกเส้น 5-3 น้ำหนัก 2 เพิ่มใน Tree



น้ำหนักรวมสะสม = 15

● กราฟที่ 1 | Prim: Spanning Tree ที่ได้

ครบทุกจุดแล้วหยุดการทำงาน | น้ำหนักเท่ากันเลือกคู่ปลายเส้นที่มีเลขน้อยก่อน



รอบ	เส้น	น้ำหนัก	สะสม
1	1-4	3	3
2	1-2	5	8
3	2-0	1	9
4	2-5	4	13
5	5-3	2	15

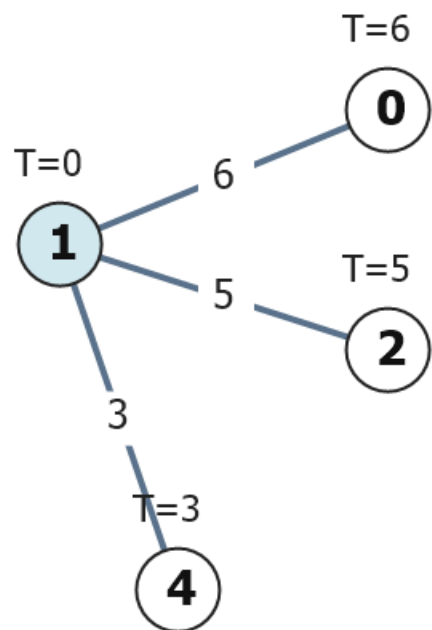
$$\text{น้ำหนักรวม} = 3 + 5 + 1 + 4 + 2 = 15$$

Tree มี 5 เส้น เชื่อมทุกจุดและไม่มีวงจร

กราฟที่ 1 | 6. Shortest Path (Dijkstra)

รอบที่ 1 | T = น้ำหนักสะสมจากจุด 1 | เลือกจุดนอก Tree ที่มี T น้อยที่สุด

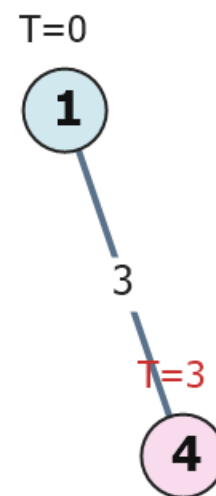
(1) เส้นทางที่พิจารณา: คำนวณ T



ค่า T ของจุดนอก Tree ที่ไปถึงได้:

0: 6 | 2: 5 | 4: 3

(2) เพิ่มจุด 4: $T = 3$



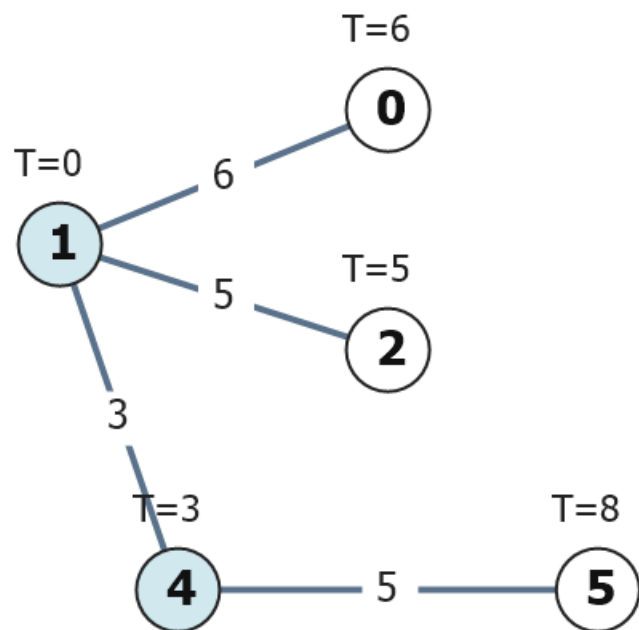
ผ่านเส้น 1-4: $0 + 3 = 3$

Tree = {1, 4}

กราฟที่ 1 | 6. Shortest Path (Dijkstra)

รอบที่ 2 | T = น้ำหนักสะสมจากจุด 1 | เลือกจุดนอก Tree ที่มี T น้อยที่สุด

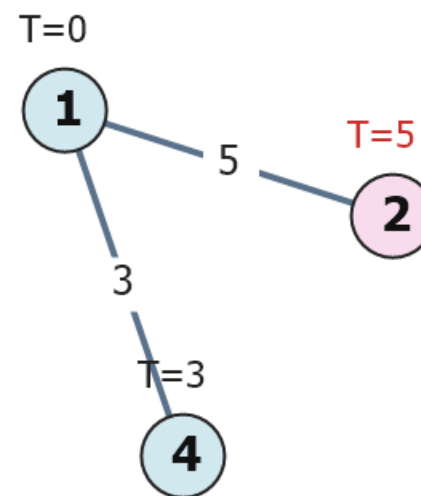
(1) เส้นทางที่พิจารณา: คำนวณ T



ค่า T ของจุดนอก Tree ที่ไปถึงได้:

0: 6 | 2: 5 | 5: 8

(2) เพิ่มจุด 2: $T = 5$



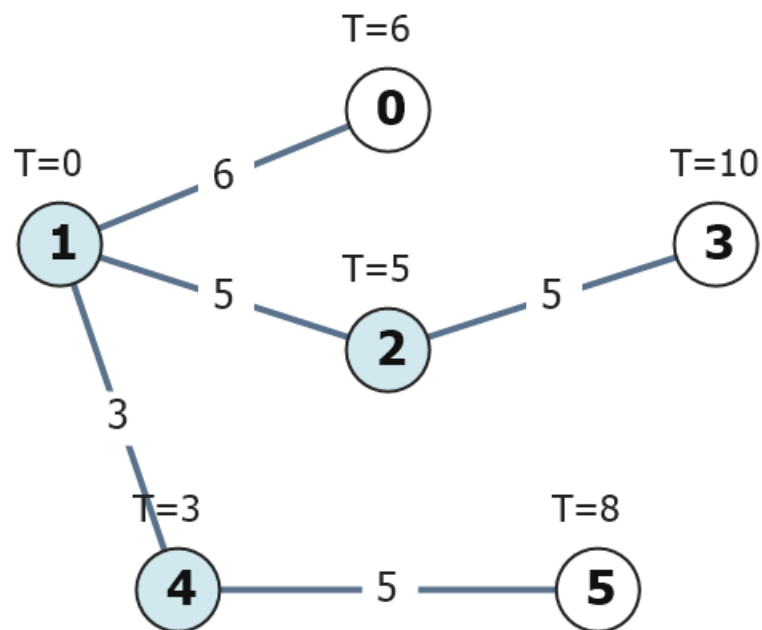
ผ่านเส้น 1-2: $0 + 5 = 5$

Tree = {1, 2, 4}

กราฟที่ 1 | 6. Shortest Path (Dijkstra)

รอบที่ 3 | T = น้ำหนักสะสมจากจุด 1 | เลือกจุดนอก Tree ที่มี T น้อยที่สุด

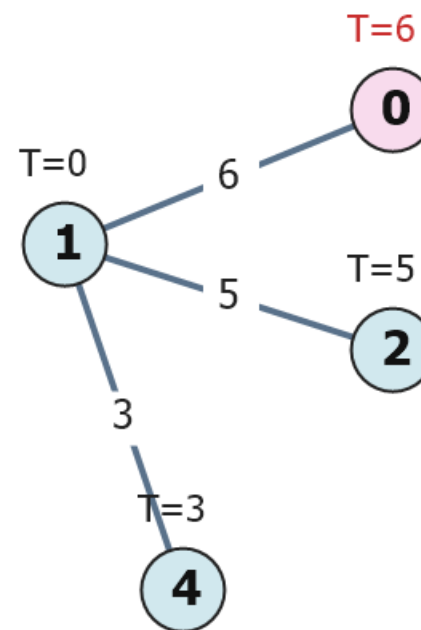
(1) เส้นทางที่พิจารณา: คำนวณ T



ค่า T ของจุดนอก Tree ที่ไปถึงได้:

0: 6 | 3: 10 | 5: 8

(2) เพิ่มจุด 0: $T = 6$



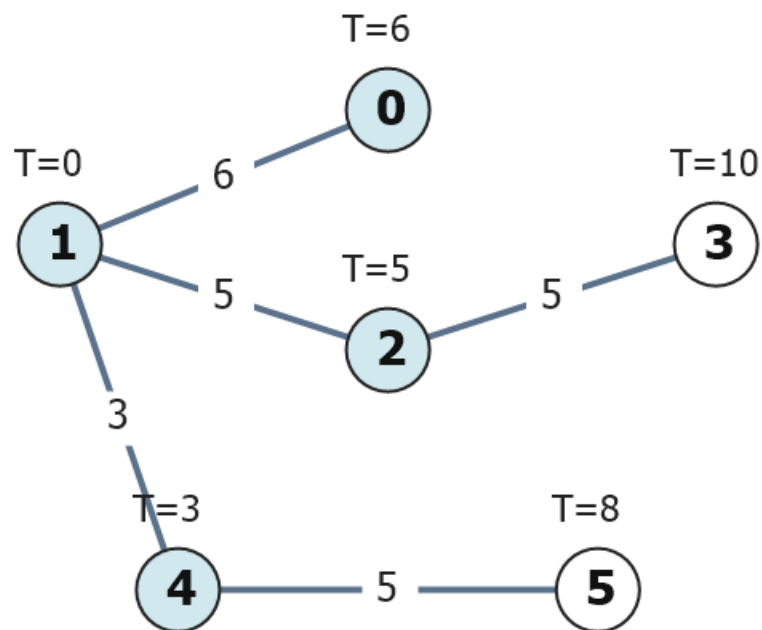
ผ่านเส้น 1-0: $0 + 6 = 6$

Tree = {0, 1, 2, 4}

กราฟที่ 1 | 6. Shortest Path (Dijkstra)

รอบที่ 4 | T = น้ำหนักสะสมจากจุด 1 | เลือกจุดนอก Tree ที่มี T น้อยที่สุด

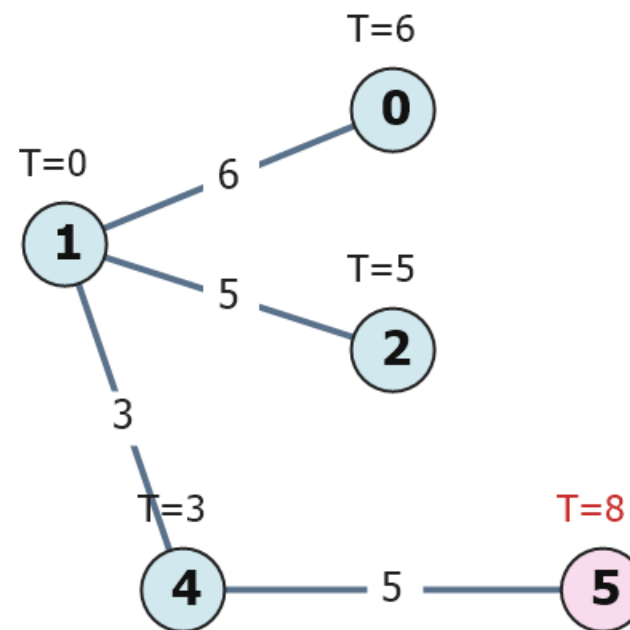
(1) เส้นทางที่พิจารณา: คำนวณ T



ค่า T ของจุดนอก Tree ที่ไปถึงได้:

3: 10 | 5: 8

(2) เพิ่มจุด 5: $T = 8$



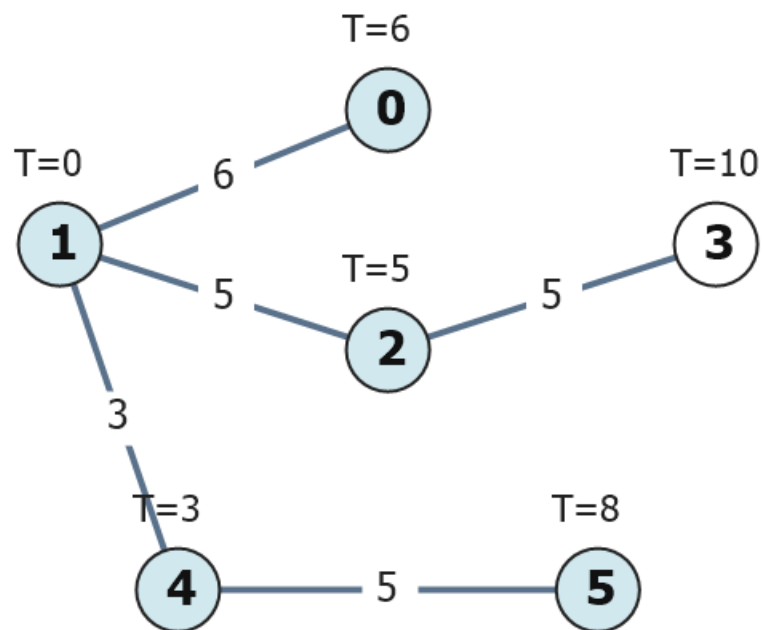
ผ่านเส้น 4-5: $3 + 5 = 8$

Tree = {0, 1, 2, 4, 5}

กราฟที่ 1 | 6. Shortest Path (Dijkstra)

รอบที่ 5 | T = น้ำหนักสะสมจากจุด 1 | เลือกจุดนอก Tree ที่มี T น้อยที่สุด

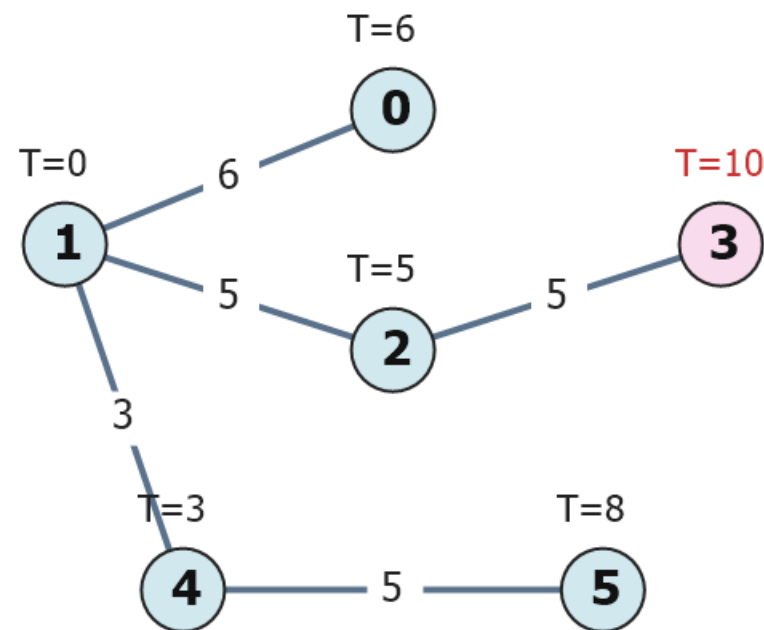
(1) เส้นทางที่พิจารณา: คำนวณ T



ค่า T ของจุดนอก Tree ที่ไปถึงได้:

3: 10

(2) เพิ่มจุด 3: $T = 10$



ผ่านเส้น 2-3: $5 + 5 = 10$

Tree = {0, 1, 2, 3, 4, 5}

● กราฟที่ 1 | Dijkstra: ตารางน้ำหนักสะสม

เริ่มด้วยจุด 1 ใน Tree | $T(1) = 0$ | ช่องตารางเป็นค่าหลังอัปเดตเพื่อนบ้านของจุดที่เพิ่ม

รอบ	เพิ่มจุด	T(0)	T(1)	T(2)	T(3)	T(4)	T(5)
เริ่ม	1	6	0	5	∞	3	∞
1	4	6	0	5	∞	3	8
2	2	6	0	5	10	3	8
3	0	6	0	5	10	3	8
4	5	6	0	5	10	3	8
5	3	6	0	5	10	3	8

ค่าเท่ากันเลือกจุดเลขน้อยก่อน | เส้นทางใหม่เท่าค่าเดิมให้เก็บจุดก่อนหน้าเดิม

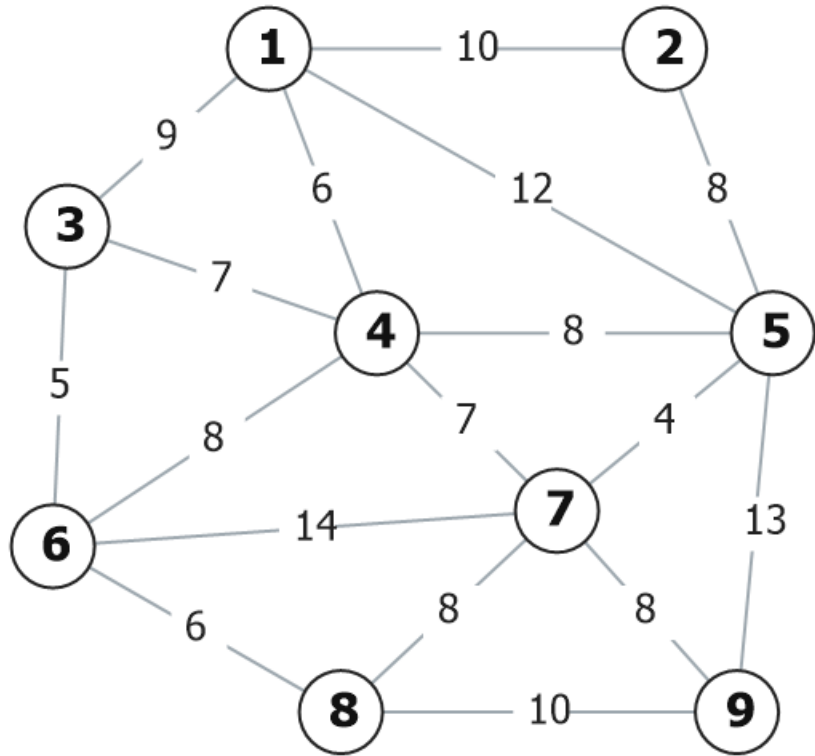
● กราฟที่ 1 | Dijkstra: เส้นทางสั้นที่สุดจากจุด 1

ผลลัพธ์มีทั้งเส้นทางและน้ำหนักสะสมตามโจทย์

ปลายทาง	เส้นทางจากจุด 1	น้ำหนักสะสม
0	1 → 0	6
1	1	0
2	1 → 2	5
3	1 → 2 → 3	10
4	1 → 4	3
5	1 → 4 → 5	8

● กราฟที่ 2 | ข้อมูลและข้อตกลง

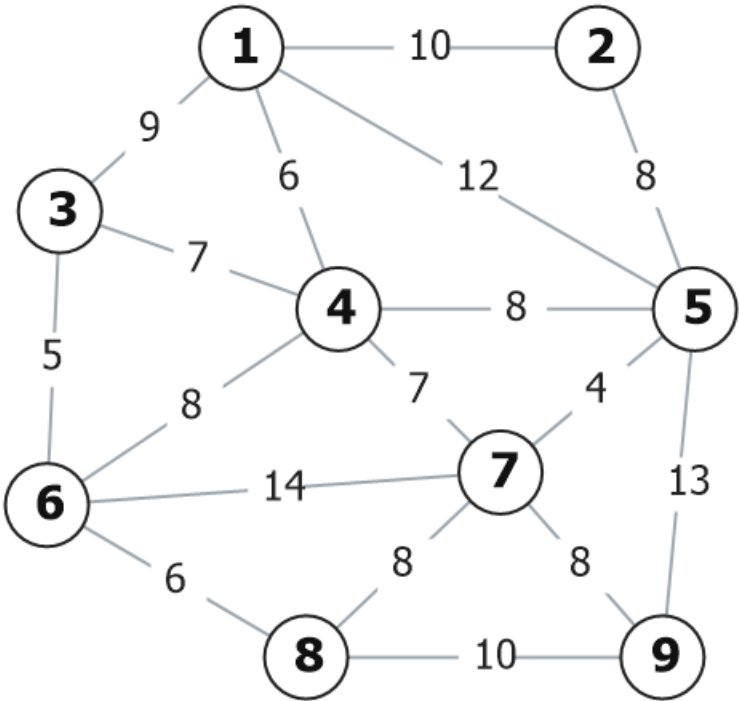
กราฟไม่มีทิศทางและมีน้ำหนัก | เริ่มจากจุด 1 ตามโจทย์



1. Adjacency Matrix ใช้น้ำหนักแทนค่า 1
2. Adjacency List แยกจุด / น้ำหนัก / ตัวชี้
3. DFS ใช้สแตก เลือกเพื่อนบ้านเลขน้อยก่อน
4. BFS ใช้คิว เลือกเพื่อนบ้านเลขน้อยก่อน
5. MST ใช้ Prim เลือกเส้นน้ำหนักน้อยสุด
6. Shortest Path ใช้ Dijkstra เลือกน้ำหนักสะสม

● กราฟที่ 2 | 1. Adjacency Matrix

กราฟมีน้ำหนัก: ใส่ น้ำหนักเมื่อมีเส้นเชื่อม และใส่ 0 เมื่อไม่มีเส้นเชื่อม

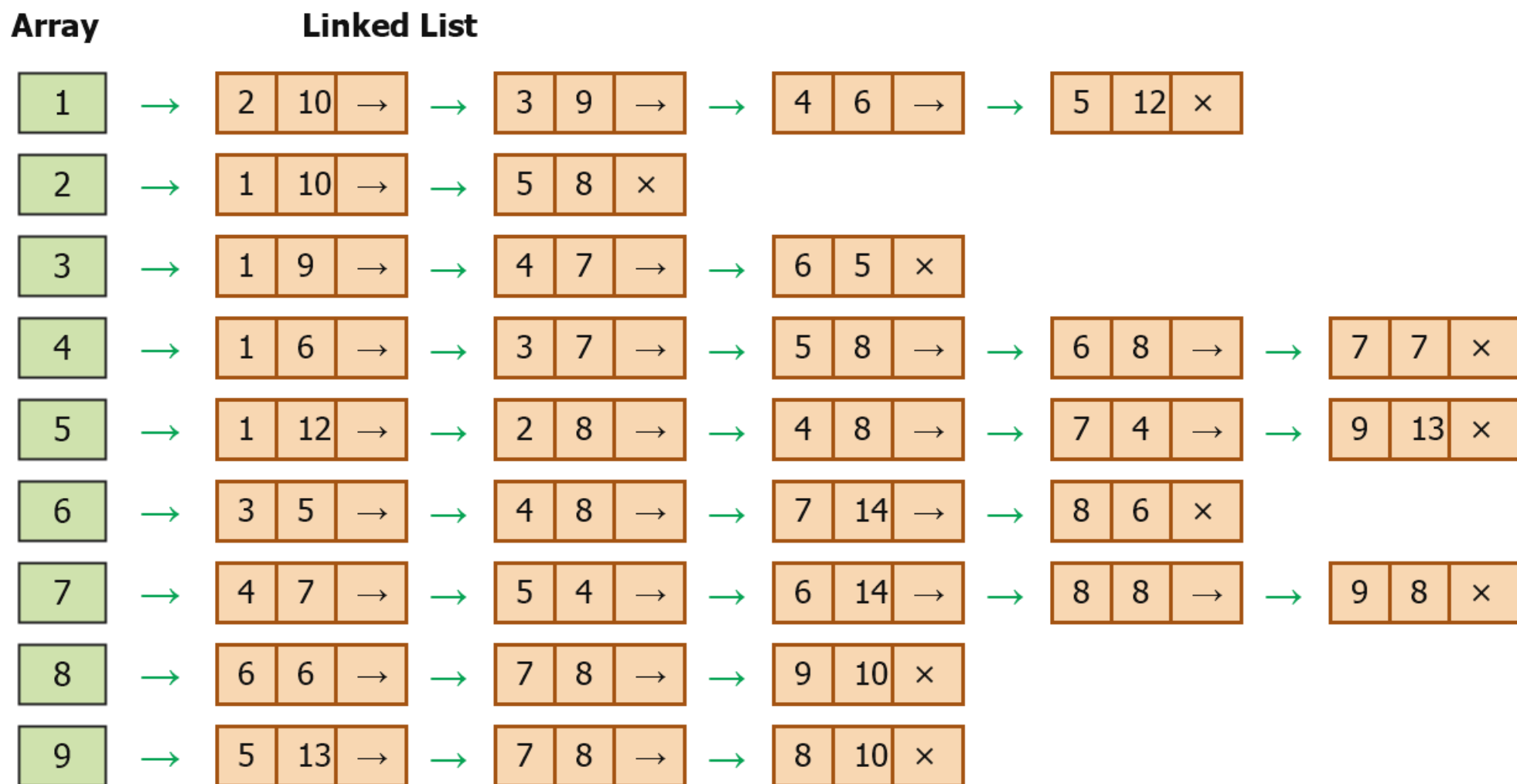


From / To	1	2	3	4	5	6	7	8	9
1	0	10	9	6	12	0	0	0	0
2	10	0	0	0	8	0	0	0	0
3	9	0	0	7	0	5	0	0	0
4	6	0	7	0	8	8	7	0	0
5	12	8	0	8	0	0	4	0	13
6	0	0	5	8	0	0	14	6	0
7	0	0	0	7	4	14	0	8	8
8	0	0	0	0	0	6	8	0	10
9	0	0	0	0	13	0	8	10	0

เมทริกซ์สมมาตร เพราะเส้นเชื่อมไม่มีทิศทาง

● กราฟที่ 2 | 2. Adjacency List

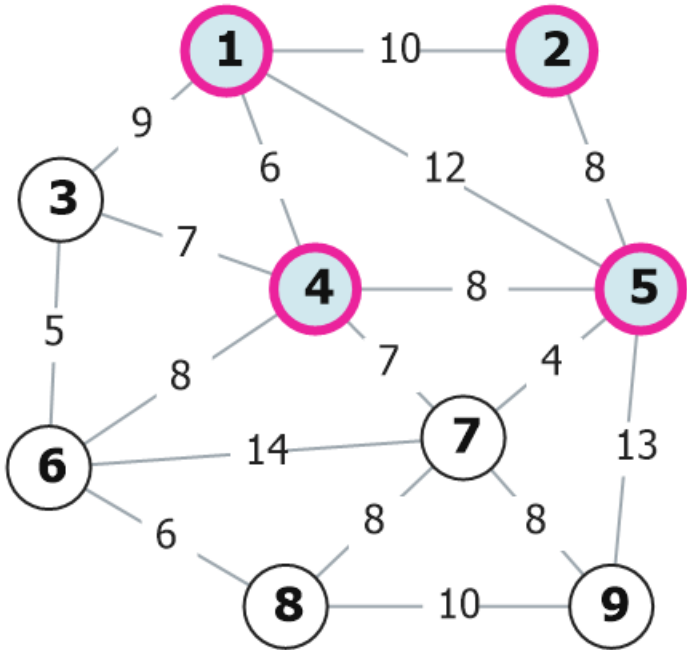
ช่องต้นทาง (Array) ชี้ไปยังลิงก์ลิสต์ | แต่ละโหนดมี [จุดข้างเคียง | น้ำหนัก | ตัวชี้]



× = NULL | มี 17 เส้น จึงมีโหนดในลิสต์ทั้งหมด 34 โหนด

กราฟที่ 2 | Depth-first traversal: Push

เลือกเพื่อนบ้านเลขน้อยก่อน | เยี่ยมชมแล้ว Push | ไม่มีเพื่อนบ้านใหม่ให้ Pop จนสแตกว่าง



Push 1

Path:
1

สแตกล้างทำงาน
Top ↑



Push 2

Path:
1 → 2

สแตกล้างทำงาน
Top ↑



Push 5

Path:
1 → 2 → 5

สแตกล้างทำงาน
Top ↑



Push 4

Path:
1 → 2 → 5 → 4

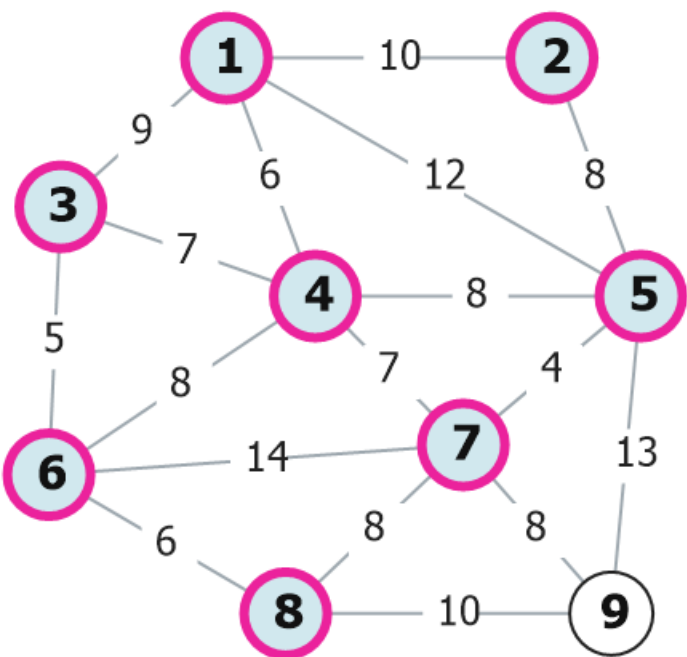
สแตกล้างทำงาน
Top ↑



Path = ลำดับเยี่ยมชม ไม่ใช่การนำจุดที่ย้อนกลับมาเขียนซ้ำ

กราฟที่ 2 | Depth-first traversal: Push

เลือกเพื่อนบ้านเลขน้อยก่อน | เยี่ยมชมแล้ว Push | ไม่มีเพื่อนบ้านใหม่ให้ Pop จนสแตกว่าง



Push 3

Path:
1 → 2 → 5 → 4
3

สแตกล้างทำงาน
Top ↑

3
4
5
2
1

Push 6

Path:
1 → 2 → 5 → 4
3 → 6

สแตกล้างทำงาน
Top ↑

6
3
4
5
2
1

Push 7

Path:
1 → 2 → 5 → 4
3 → 6 → 7

สแตกล้างทำงาน
Top ↑

7
6
3
4
5
2
1

Push 8

Path:
1 → 2 → 5 → 4
3 → 6 → 7 → 8

สแตกล้างทำงาน
Top ↑

8
7
6
3
4
5
2
1

Path = ลำดับเยี่ยมชม ไม่ใช่การนำจุดที่ย้อนกลับมาเขียนซ้ำ

● กราฟที่ 2 | Depth-first traversal: Push

เลือกเพื่อนบ้านเลขน้อยก่อน | เยี่ยมชมแล้ว Push | ไม่มีเพื่อนบ้านใหม่ให้ Pop จนสแตกว่าง

Push 9

Path:

1 → 2 → 5 → 4

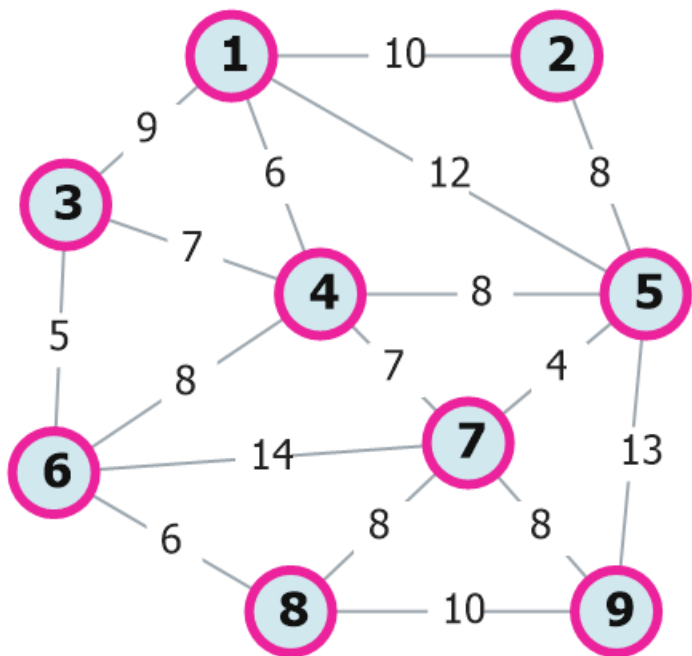
3 → 6 → 7 → 8

9

สแตกหลังทำงาน

Top ↑

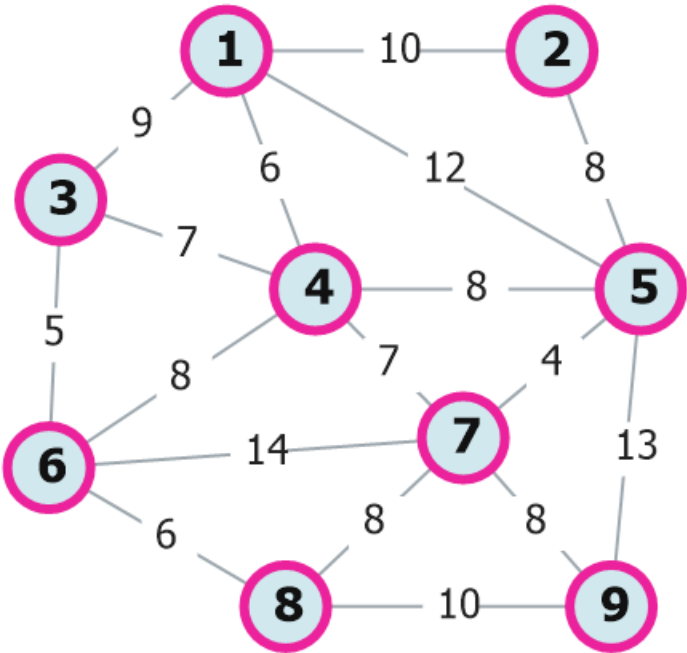
9
8
7
6
3
4
5
2
1



Path = ลำดับเยี่ยมชม ไม่ใช่การนำจุดที่ย้อนกลับมาเขียนซ้ำ

กราฟที่ 2 | Depth-first traversal: Pop และย้อนกลับ

เลือกเพื่อนบ้านเลขน้อยก่อน | เยี่ยมชมแล้ว Push | ไม่มีเพื่อนบ้านใหม่ให้ Pop จนสแตกว่าง



Pop 9

Path:
1 → 2 → 5 → 4
3 → 6 → 7 → 8
9

สแตกล้างทำงาน
Top ↑

8
7
6
3
4
5
2
1

Pop 8

Path:
1 → 2 → 5 → 4
3 → 6 → 7 → 8
9

สแตกล้างทำงาน
Top ↑

7
6
3
4
5
2
1

Pop 7

Path:
1 → 2 → 5 → 4
3 → 6 → 7 → 8
9

สแตกล้างทำงาน
Top ↑

6
3
4
5
2
1

Pop 6

Path:
1 → 2 → 5 → 4
3 → 6 → 7 → 8
9

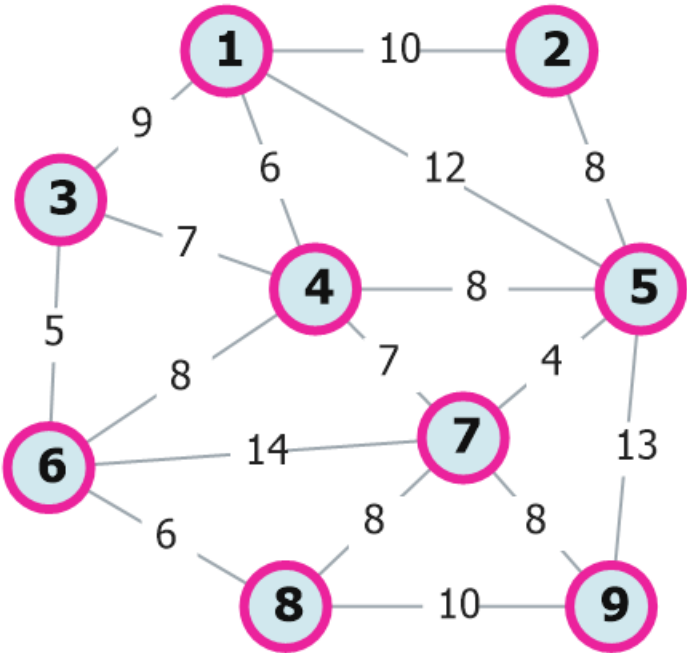
สแตกล้างทำงาน
Top ↑

3
4
5
2
1

Path = ลำดับเยี่ยมชม ไม่ใช่การนำจุดที่ย้อนกลับมาเขียนซ้ำ

กราฟที่ 2 | Depth-first traversal: Pop และย้อนกลับ

เลือกเพื่อนบ้านเลขน้อยก่อน | เยี่ยมชมแล้ว Push | ไม่มีเพื่อนบ้านใหม่ให้ Pop จนสแตกว่าง



Pop 3

Path:
1 → 2 → 5 → 4
3 → 6 → 7 → 8
9

สแตกล้างทำงาน
Top ↑



Pop 4

Path:
1 → 2 → 5 → 4
3 → 6 → 7 → 8
9

สแตกล้างทำงาน
Top ↑



Pop 5

Path:
1 → 2 → 5 → 4
3 → 6 → 7 → 8
9

สแตกล้างทำงาน
Top ↑



Pop 2

Path:
1 → 2 → 5 → 4
3 → 6 → 7 → 8
9

สแตกล้างทำงาน
Top ↑



Path = ลำดับเยี่ยมชม ไม่ใช่การนำจุดที่ย้อนกลับมาเขียนซ้ำ

กราฟที่ 2 | Depth-first traversal: Pop และย้อนกลับ

เลือกเพื่อนบ้านเลขน้อยก่อน | เยี่ยมชมแล้ว Push | ไม่มีเพื่อนบ้านใหม่ให้ Pop จนสแตกว่าง

Pop 1

Path:

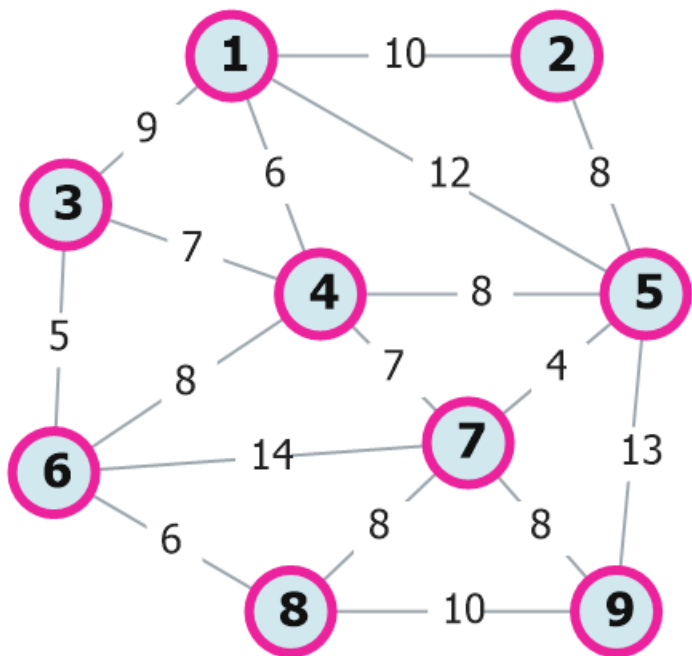
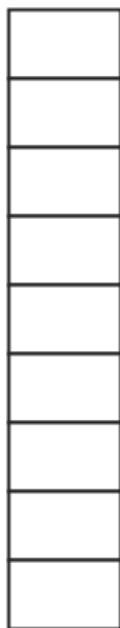
$1 \rightarrow 2 \rightarrow 5 \rightarrow 4$

$3 \rightarrow 6 \rightarrow 7 \rightarrow 8$

9

สแตกหลังทำงาน

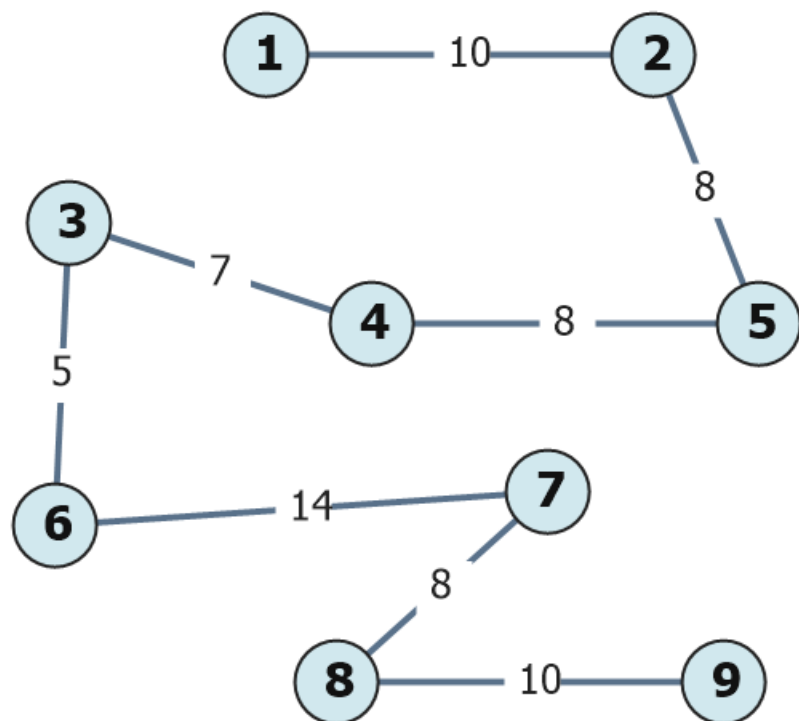
Top ↑



Path = ลำดับเยี่ยมชม ไม่ใช่การนำจุดที่ย้อนกลับมาเขียนซ้ำ

● กราฟที่ 2 | DFS: Path ที่ได้

หลัง Pop จุดเริ่มต้น สแตกว่าง จึงหยุดการทำงาน



Path:

1 → 2 → 5 → 4 → 3 → 6 → 7 → 8 → 9

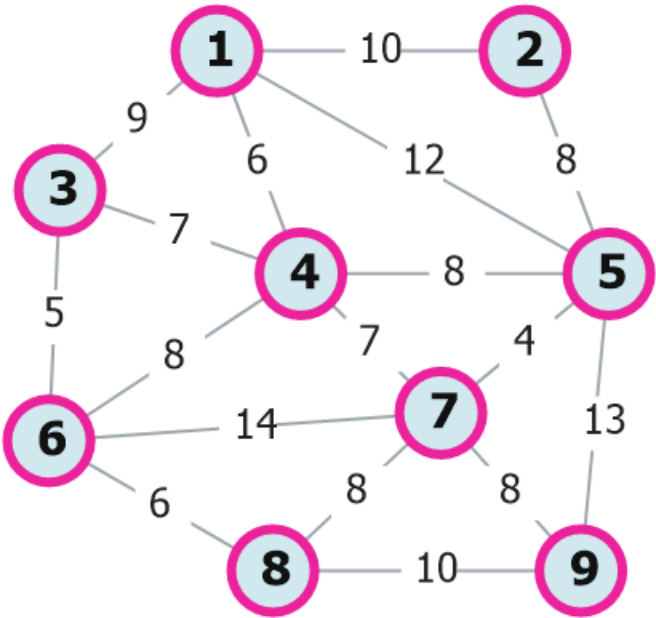
ต้นไม้ DFS:

1-2, 2-5, 5-4, 4-3

3-6, 6-7, 7-8, 8-9

กราฟที่ 2 | 4. Breadth-first traversal

เริ่มคิว [1] | ค้นพบแล้วทำเครื่องหมายทันที | เต็มท้ายคิวและนำออกจากหัวคิว



R1: นำ 1 ออกจากคิว

PATH: 1

2	3	4	5	
---	---	---	---	--

R2: นำ 2 ออกจากคิว

PATH: 1-2

3	4	5		
---	---	---	--	--

R3: นำ 3 ออกจากคิว

PATH: 1-2-3

4	5	6		
---	---	---	--	--

R4: นำ 4 ออกจากคิว

PATH: 1-2-3-4

5	6	7		
---	---	---	--	--

R5: นำ 5 ออกจากคิว

PATH: 1-2-3-4-5

6	7	9		
---	---	---	--	--

R6: นำ 6 ออกจากคิว

PATH: 1-2-3-4-5-6

7	9	8		
---	---	---	--	--

R7: นำ 7 ออกจากคิว

PATH: 1-2-3-4-5-6-7

9	8			
---	---	--	--	--

R8: นำ 9 ออกจากคิว

PATH: 1-2-3-4-5-6-7-9

8				
---	--	--	--	--

R9: นำ 8 ออกจากคิว

PATH: 1-2-3-4-5-6-7-9-8

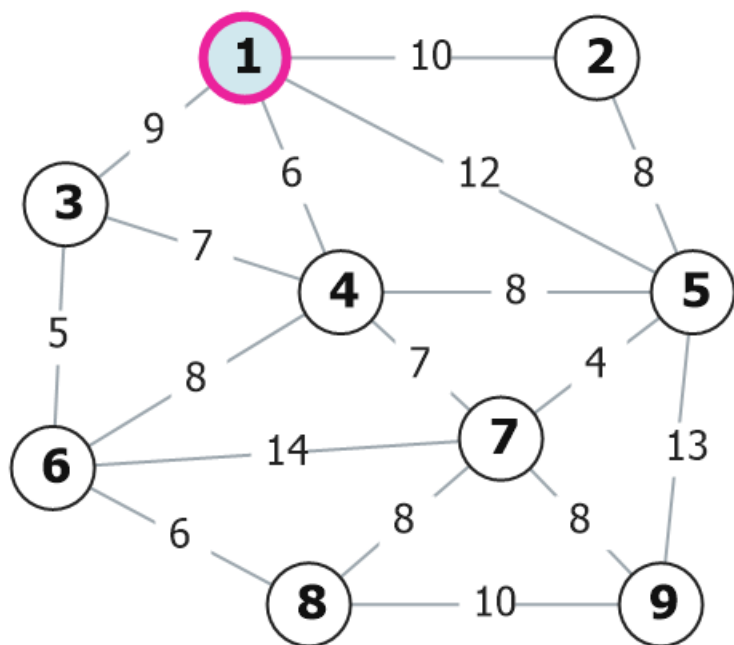
--	--	--	--	--

หัวคิวอยู่ซ้าย | คิวว่างในรอบสุดท้าย จบการทำงาน

● กราฟที่ 2 | 5. Minimum Spanning Tree (Prim)

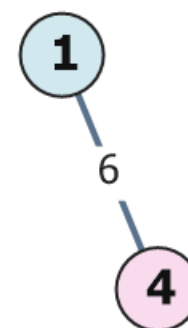
รอบที่ 1 | พิจารณาเส้นจากจุดใน Tree ไปยังจุดที่ยังไม่อยู่ใน Tree

กราฟเดิม: พิจารณาจุด 1



เส้นที่เลือกได้ (เรียงตามน้ำหนัก):
1-4(6), 1-3(9), 1-2(10), 1-5(12)

เลือกเส้น 1-4 น้ำหนัก 6 เพิ่มใน Tree

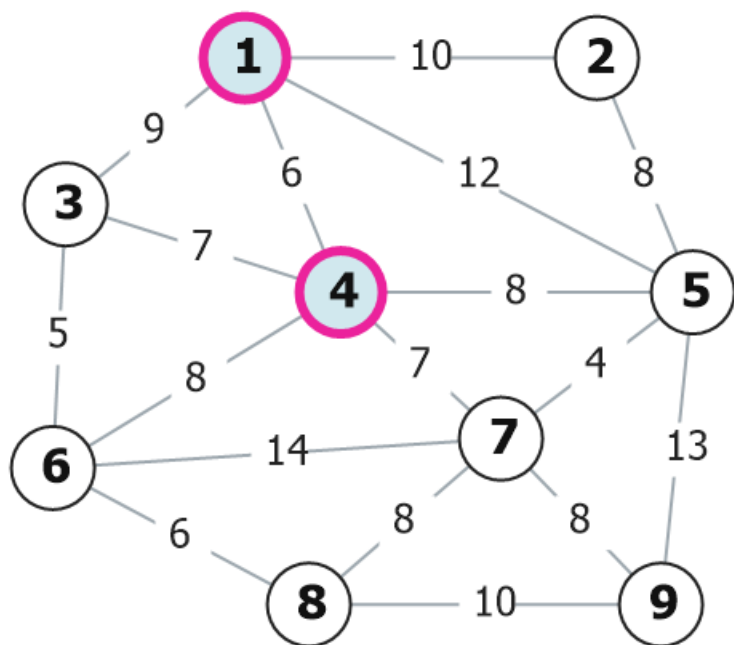


น้ำหนักรวมสะสม = 6

● กราฟที่ 2 | 5. Minimum Spanning Tree (Prim)

รอบที่ 2 | พิจารณาเส้นจากจุดใน Tree ไปยังจุดที่ยังไม่อยู่ใน Tree

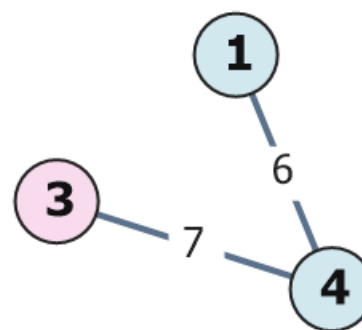
กราฟเดิม: พิจารณาจุด 1, 4



เส้นที่เลือกได้ (เรียงตามน้ำหนัก):

3-4(7), 4-7(7), 4-5(8), 4-6(8), 1-3(9), 1-2(10), 1-5(12)

เลือกเส้น 4-3 น้ำหนัก 7 เพิ่มใน Tree

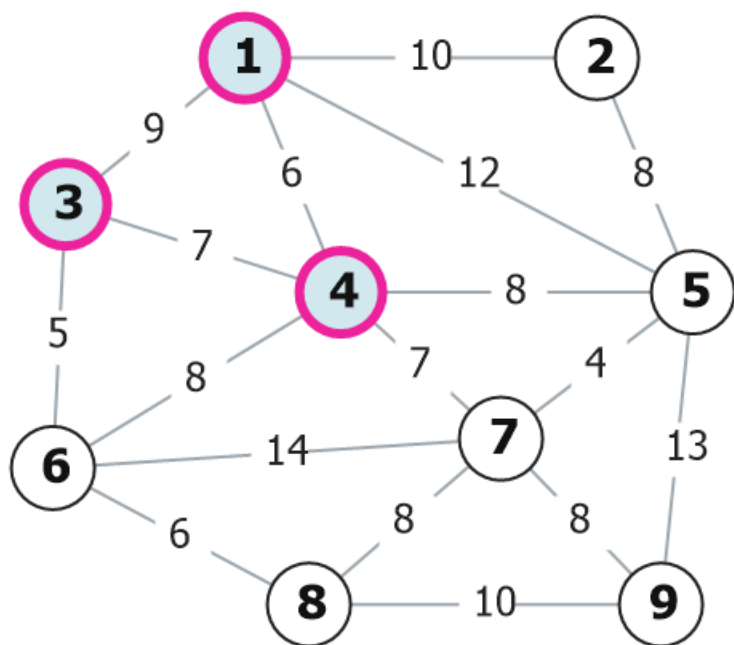


น้ำหนักรวมสะสม = 13

● กราฟที่ 2 | 5. Minimum Spanning Tree (Prim)

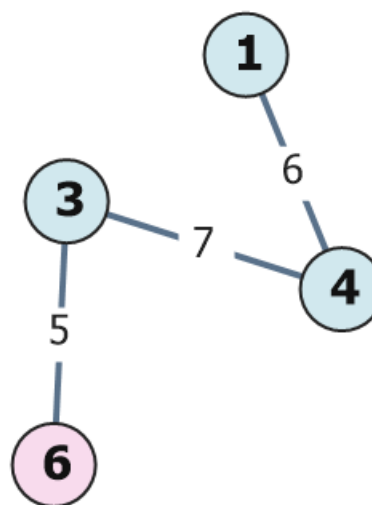
รอบที่ 3 | พิจารณาเส้นจากจุดใน Tree ไปยังจุดที่ยังไม่อยู่ใน Tree

กราฟเดิม: พิจารณาจุด 1, 3, 4



เส้นที่เลือกได้ (เรียงตามน้ำหนัก):
3-6(5), 4-7(7), 4-5(8), 4-6(8), 1-2(10), 1-5(12)

เลือกเส้น 3-6 น้ำหนัก 5 เพิ่มใน Tree

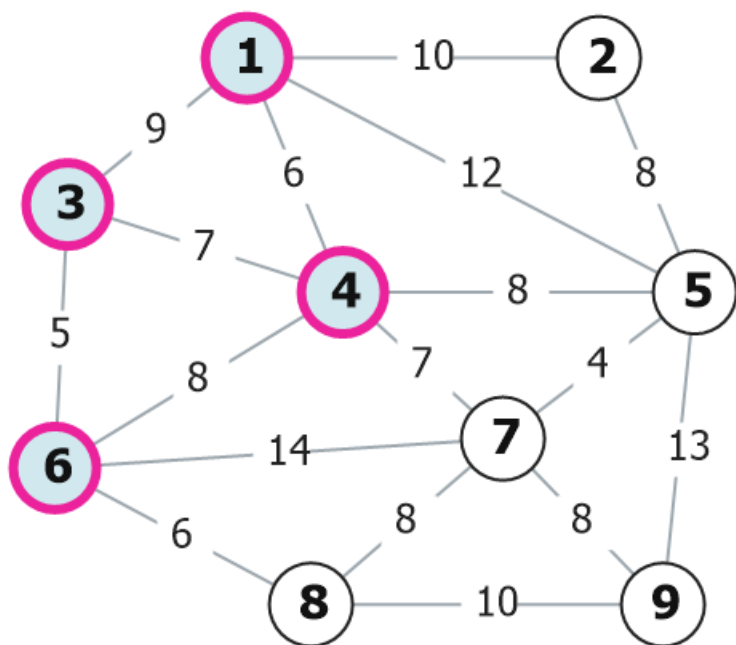


น้ำหนักรวมสะสม = 18

● กราฟที่ 2 | 5. Minimum Spanning Tree (Prim)

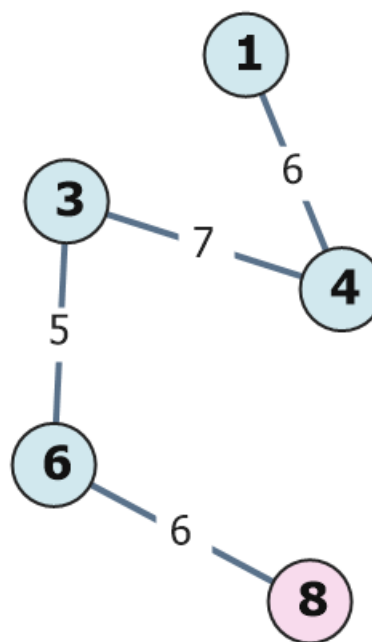
รอบที่ 4 | พิจารณาเส้นจากจุดใน Tree ไปยังจุดที่ยังไม่อยู่ใน Tree

กราฟเดิม: พิจารณาจุด 1, 3, 4, 6



เส้นที่เลือกได้ (เรียงตามน้ำหนัก):
6-8(6), 4-7(7), 4-5(8), 1-2(10), 1-5(12), 6-7(14)

เลือกเส้น 6-8 น้ำหนัก 6 เพิ่มใน Tree

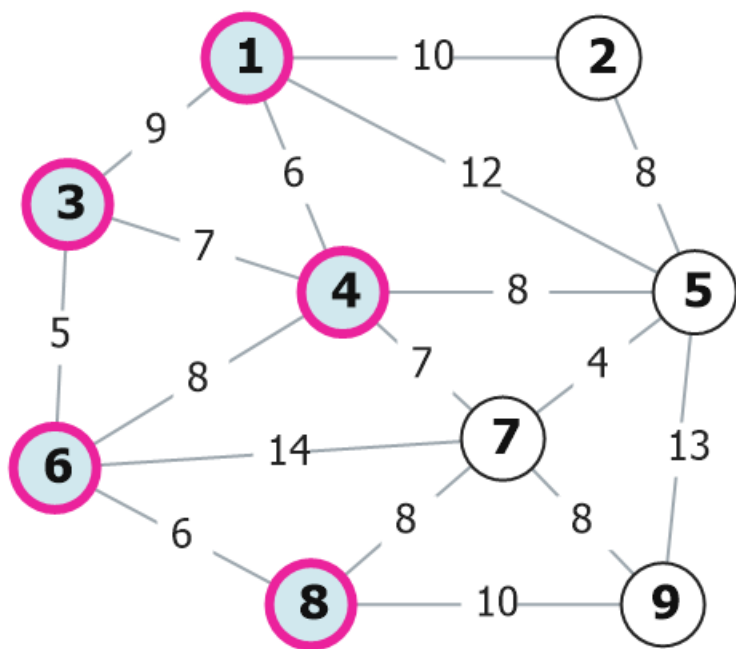


น้ำหนักรวมสะสม = 24

● กราฟที่ 2 | 5. Minimum Spanning Tree (Prim)

รอบที่ 5 | พิจารณาเส้นจากจุดใน Tree ไปยังจุดที่ยังไม่อยู่ใน Tree

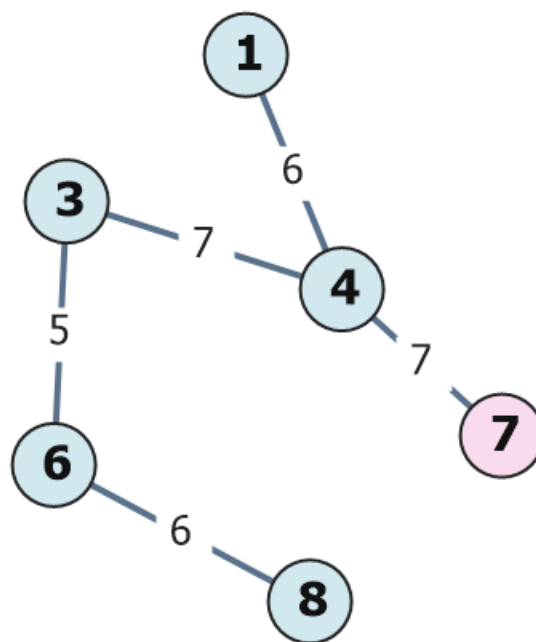
กราฟเดิม: พิจารณาจุด 1, 3, 4, 6, 8



เส้นที่เลือกได้ (เรียงตามน้ำหนัก):

4-7(7), 4-5(8), 7-8(8), 1-2(10), 8-9(10), 1-5(12), 6-7(14)

เลือกเส้น 4-7 น้ำหนัก 7 เพิ่มใน Tree

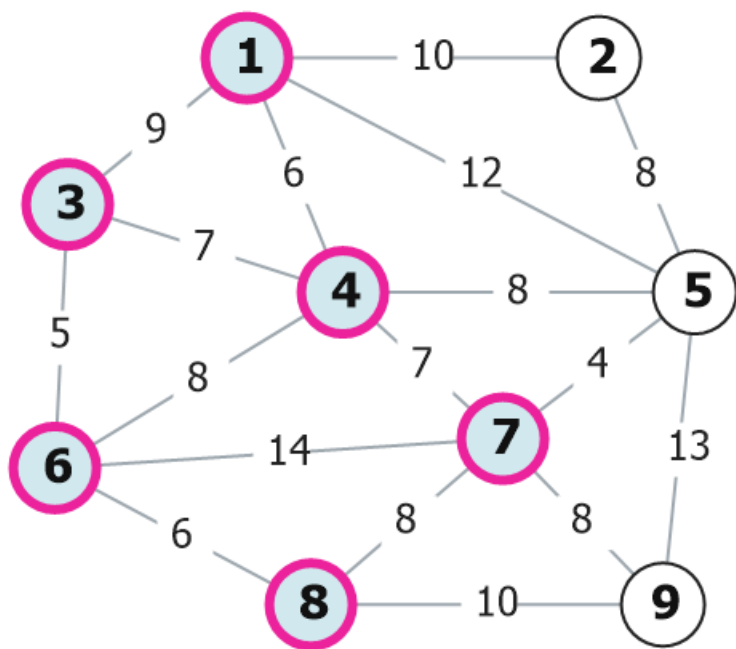


น้ำหนักรวมสะสม = 31

กราฟที่ 2 | 5. Minimum Spanning Tree (Prim)

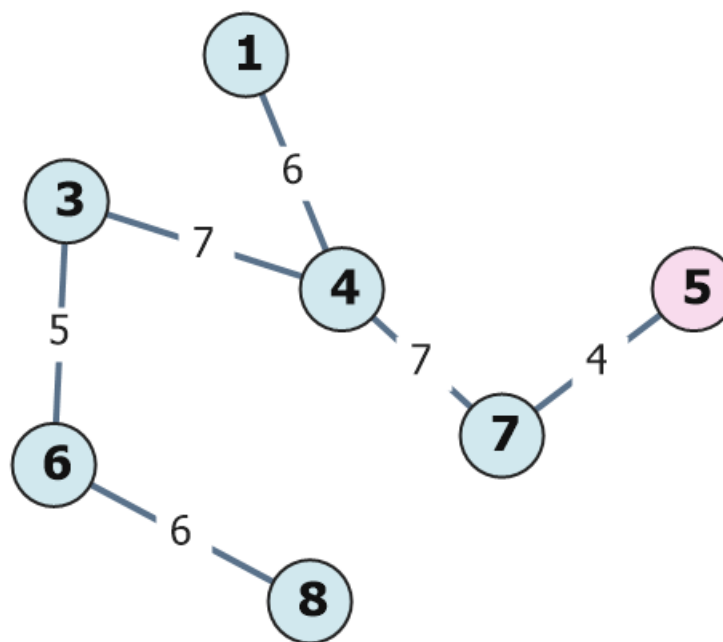
รอบที่ 6 | พิจารณาเส้นจากจุดใน Tree ไปยังจุดที่ยังไม่อยู่ใน Tree

กราฟเดิม: พิจารณาจุด 1, 3, 4, 6, 7, 8



เส้นที่เลือกได้ (เรียงตามน้ำหนัก):
5-7(4), 4-5(8), 7-9(8), 1-2(10), 8-9(10), 1-5(12)

เลือกเส้น 7-5 น้ำหนัก 4 เพิ่มใน Tree

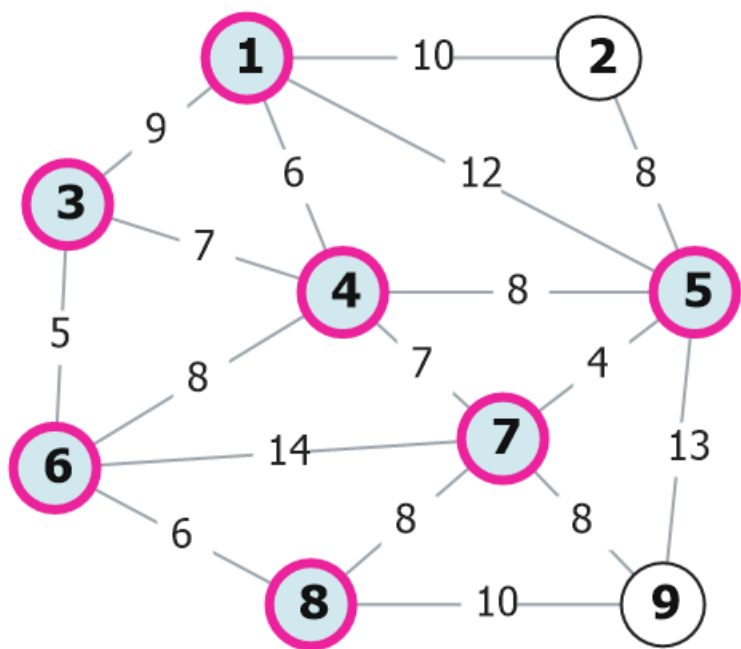


น้ำหนักรวมสะสม = 35

กราฟที่ 2 | 5. Minimum Spanning Tree (Prim)

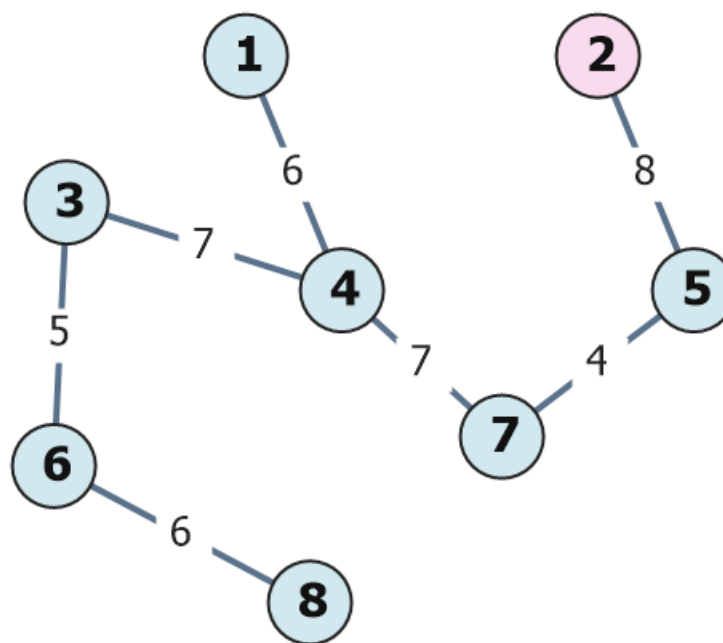
รอบที่ 7 | พิจารณาเส้นจากจุดใน Tree ไปยังจุดที่ยังไม่อยู่ใน Tree

กราฟเดิม: พิจารณาจุด 1, 3, 4, 5, 6, 7, 8



เส้นที่เลือกได้ (เรียงตามน้ำหนัก):
2-5(8), 7-9(8), 1-2(10), 8-9(10), 5-9(13)

เลือกเส้น 5-2 น้ำหนัก 8 เพิ่มใน Tree

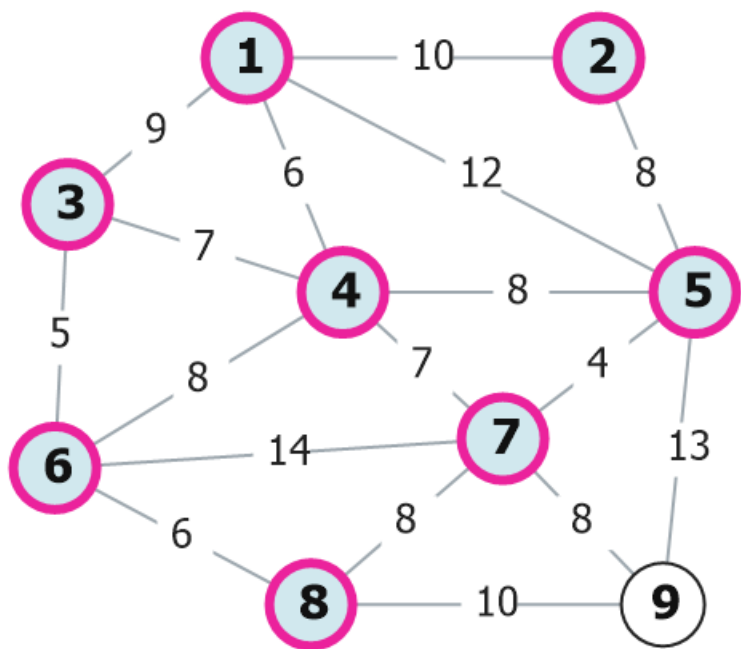


น้ำหนักรวมสะสม = 43

กราฟที่ 2 | 5. Minimum Spanning Tree (Prim)

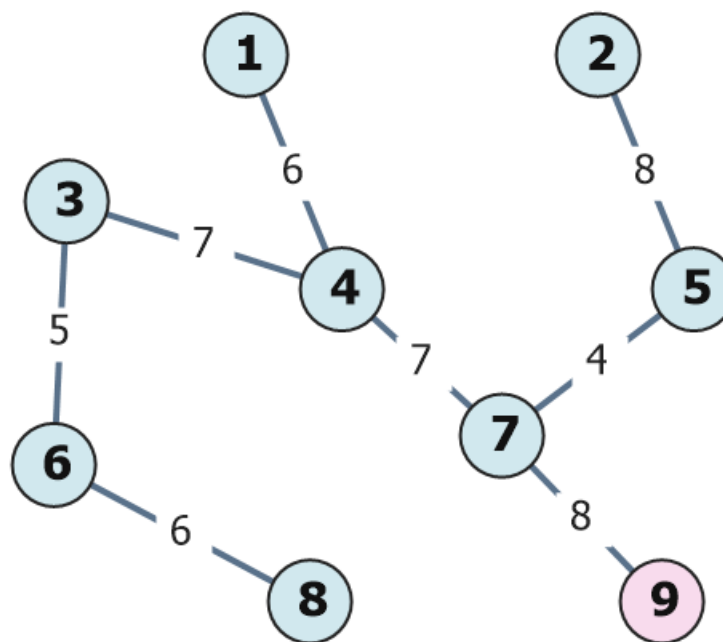
รอบที่ 8 | พิจารณาเส้นจากจุดใน Tree ไปยังจุดที่ยังไม่อยู่ใน Tree

กราฟเดิม: พิจารณาจุด 1, 2, 3, 4, 5, 6, 7, 8



เส้นที่เลือกได้ (เรียงตามน้ำหนัก):
7-9(8), 8-9(10), 5-9(13)

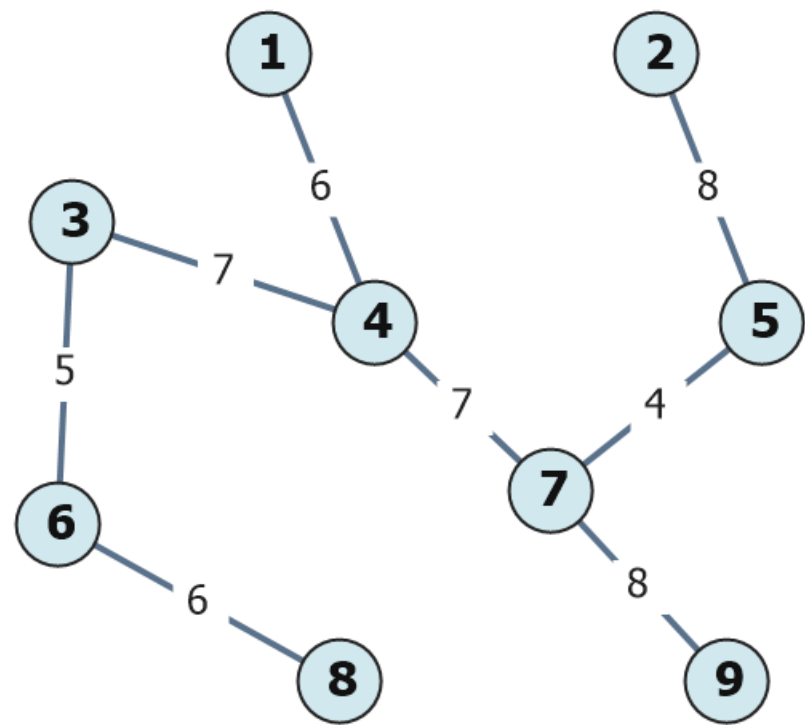
เลือกเส้น 7-9 น้ำหนัก 8 เพิ่มใน Tree



น้ำหนักรวมสะสม = 51

กราฟที่ 2 | Prim: Spanning Tree ที่ได้

ครบทุกจุดแล้วหยุดการทำงาน | น้ำหนักเท่ากันเลือกคู่ปลายเส้นที่มีเลขน้อยก่อน



รอบ	เส้น	น้ำหนัก	สะสม
1	1-4	6	6
2	4-3	7	13
3	3-6	5	18
4	6-8	6	24
5	4-7	7	31
6	7-5	4	35
7	5-2	8	43
8	7-9	8	51

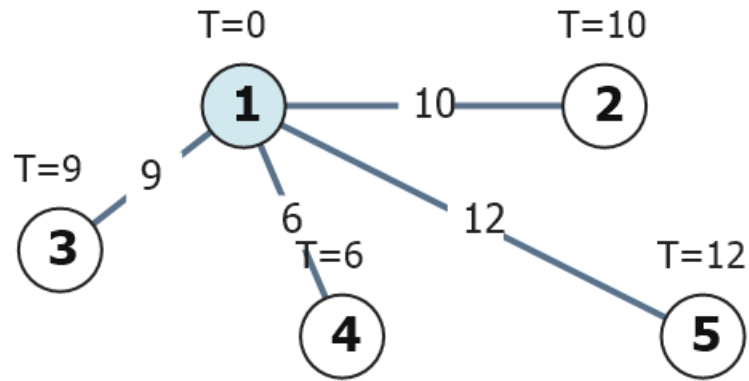
น้ำหนักรวม = 6 + 7 + 5 + 6 + 7 + 4 + 8 + 8 = 51

Tree มี 8 เส้น เชื่อมทุกจุดและไม่มีวงจร

● กราฟที่ 2 | 6. Shortest Path (Dijkstra)

รอบที่ 1 | T = น้ำหนักสะสมจากจุด 1 | เลือกจุดนอก Tree ที่มี T น้อยที่สุด

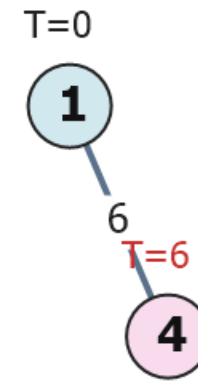
(1) เส้นทางที่พิจารณา: คำนวณ T



ค่า T ของจุดนอก Tree ที่ไปถึงได้:

2: 10 | 3: 9 | 4: 6 | 5: 12

(2) เพิ่มจุด 4: $T = 6$



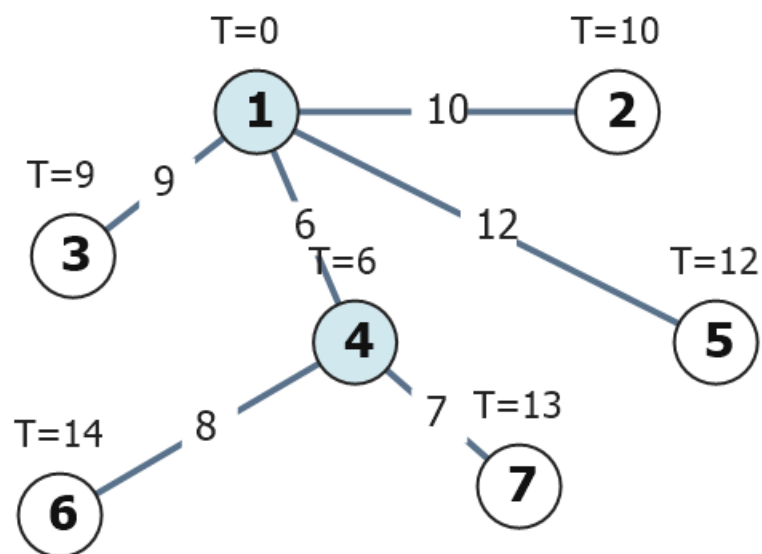
ผ่านเส้น 1-4: $0 + 6 = 6$

Tree = {1, 4}

กราฟที่ 2 | 6. Shortest Path (Dijkstra)

รอบที่ 2 | T = น้ำหนักสะสมจากจุด 1 | เลือกจุดนอก Tree ที่มี T น้อยที่สุด

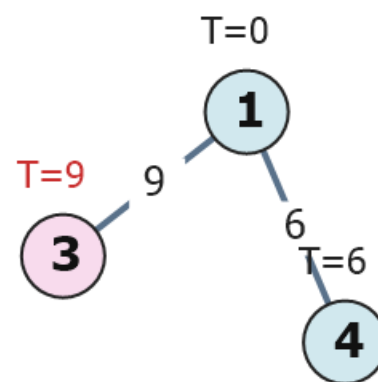
(1) เส้นทางที่พิจารณา: ค่ารวม T



ค่า T ของจุดนอก Tree ที่ไปถึงได้:

2: 10 | 3: 9 | 5: 12 | 6: 14 | 7: 13

(2) เพิ่มจุด 3: $T = 9$



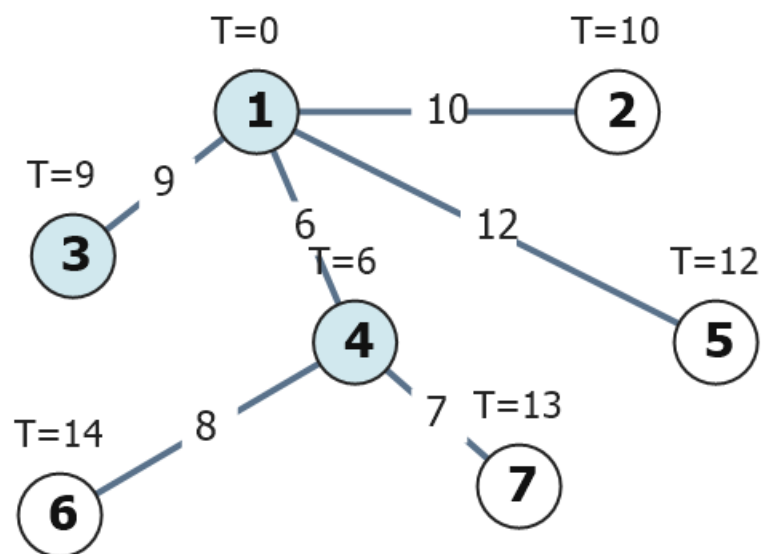
ผ่านเส้น 1-3: $0 + 9 = 9$

Tree = {1, 3, 4}

กราฟที่ 2 | 6. Shortest Path (Dijkstra)

รอบที่ 3 | T = น้ำหนักสะสมจากจุด 1 | เลือกจุดนอก Tree ที่มี T น้อยที่สุด

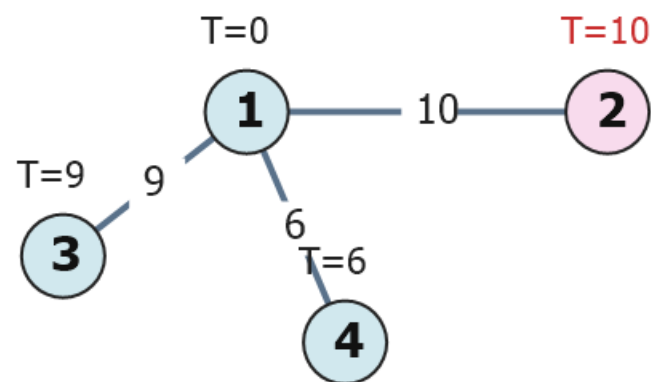
(1) เส้นทางที่พิจารณา: คำนวณ T



ค่า T ของจุดนอก Tree ที่ไปถึงได้:

2: 10 | 5: 12 | 6: 14 | 7: 13

(2) เพิ่มจุด 2: $T = 10$



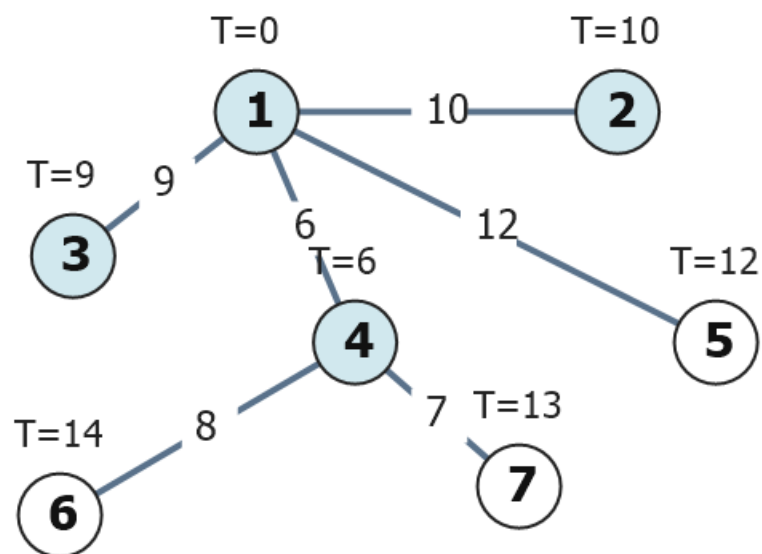
ผ่านเส้น 1-2: $0 + 10 = 10$

Tree = {1, 2, 3, 4}

กราฟที่ 2 | 6. Shortest Path (Dijkstra)

รอบที่ 4 | T = น้ำหนักสะสมจากจุด 1 | เลือกจุดนอก Tree ที่มี T น้อยที่สุด

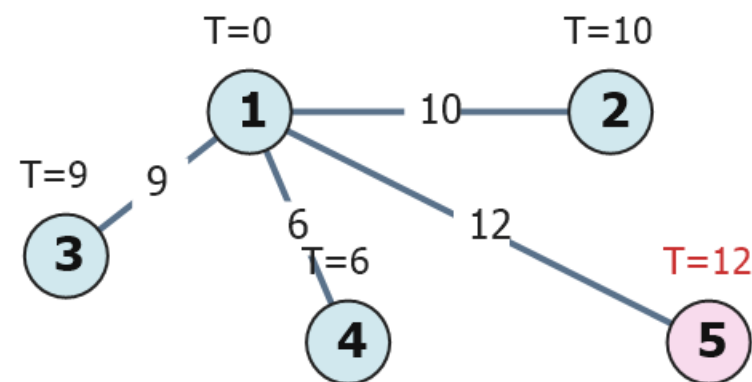
(1) เส้นทางที่พิจารณา: คำนวณ T



ค่า T ของจุดนอก Tree ที่ไปถึงได้:

5: 12 | 6: 14 | 7: 13

(2) เพิ่มจุด 5: $T = 12$



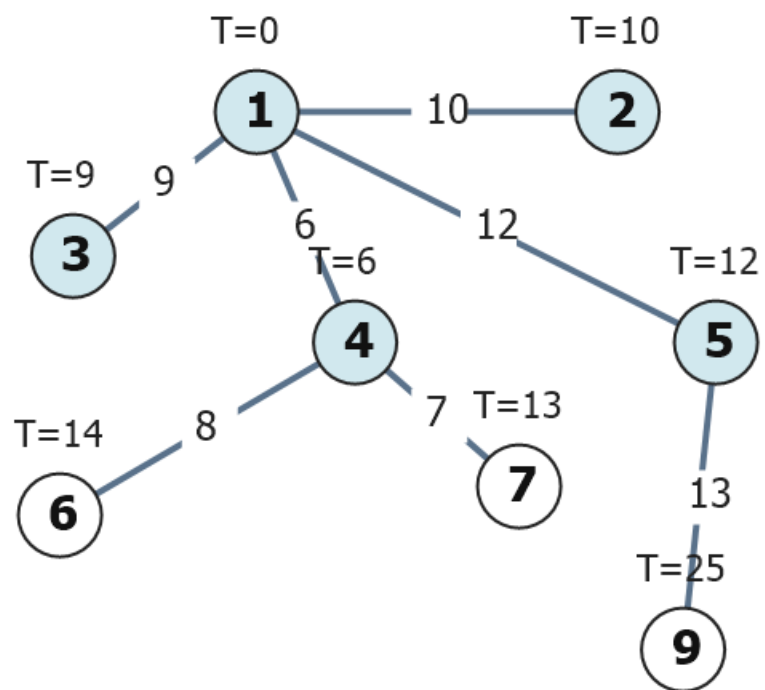
ผ่านเส้น 1-5: $0 + 12 = 12$

Tree = {1, 2, 3, 4, 5}

กราฟที่ 2 | 6. Shortest Path (Dijkstra)

รอบที่ 5 | T = น้ำหนักสะสมจากจุด 1 | เลือกจุดนอก Tree ที่มี T น้อยที่สุด

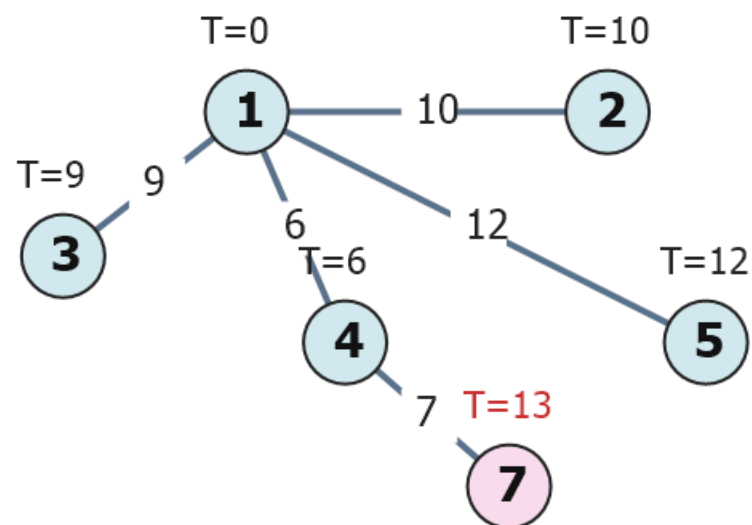
(1) เส้นทางที่พิจารณา: คำนวณ T



ค่า T ของจุดนอก Tree ที่ไปถึงได้:

6: 14 | 7: 13 | 9: 25

(2) เพิ่มจุด 7: $T = 13$



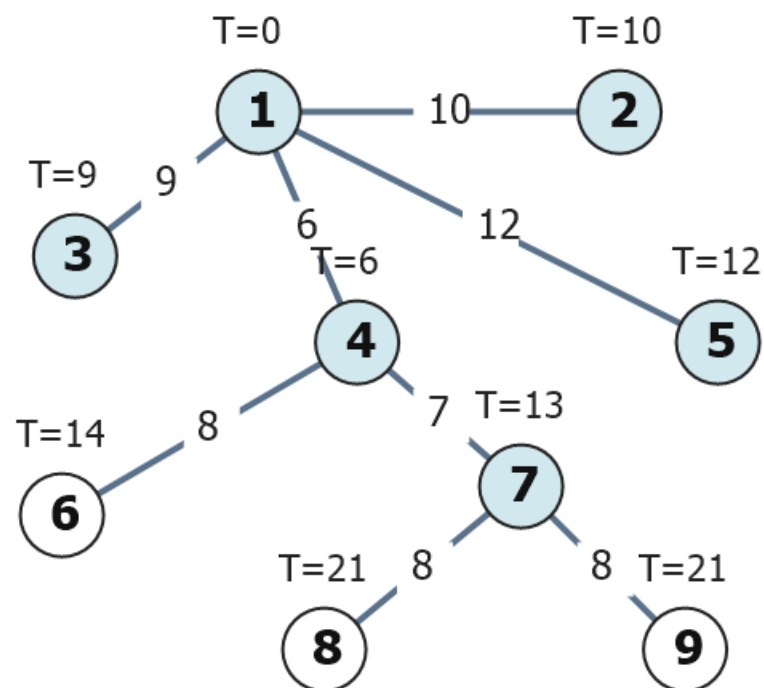
ผ่านเส้น 4-7: $6 + 7 = 13$

Tree = {1, 2, 3, 4, 5, 7}

กราฟที่ 2 | 6. Shortest Path (Dijkstra)

รอบที่ 6 | T = น้ำหนักสะสมจากจุด 1 | เลือกจุดนอก Tree ที่มี T น้อยที่สุด

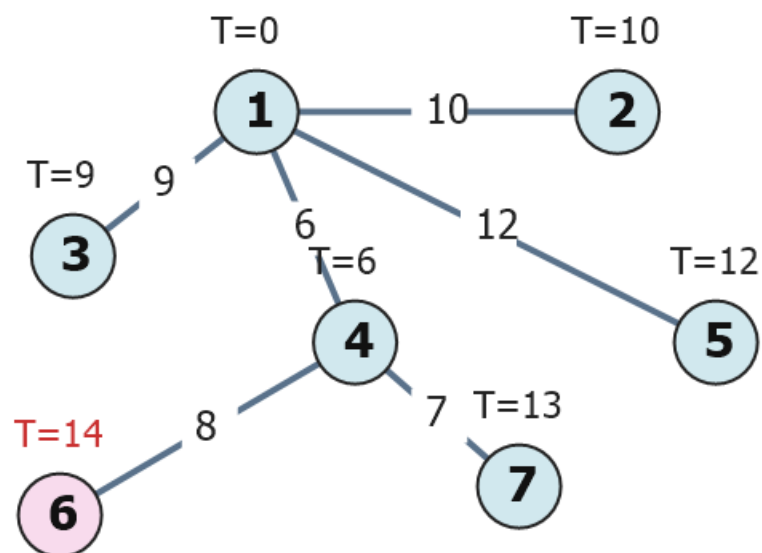
(1) เส้นทางที่พิจารณา: คำนวณ T



ค่า T ของจุดนอก Tree ที่ไปถึงได้:

6: 14 | 8: 21 | 9: 21

(2) เพิ่มจุด 6: $T = 14$



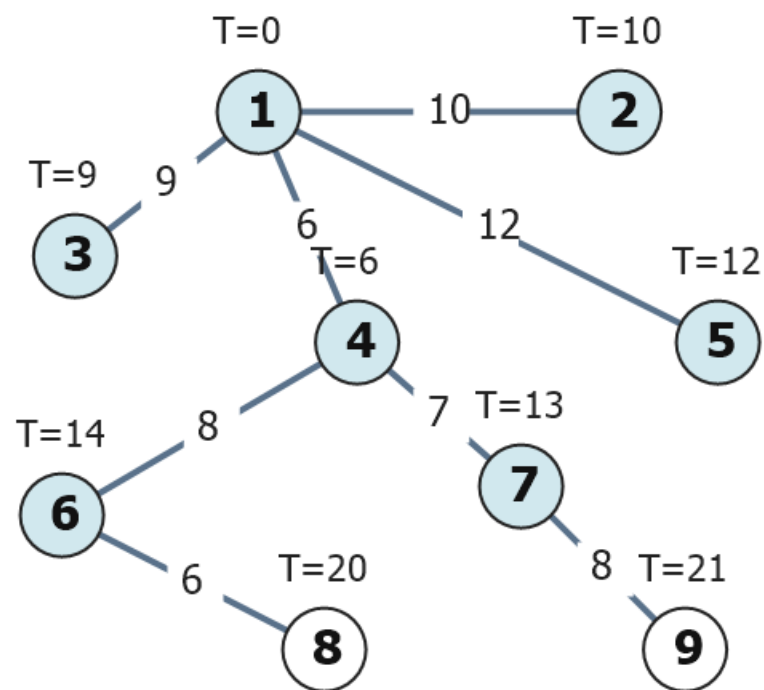
ผ่านเส้น 4-6: $6 + 8 = 14$

Tree = {1, 2, 3, 4, 5, 6, 7}

กราฟที่ 2 | 6. Shortest Path (Dijkstra)

รอบที่ 7 | T = น้ำหนักสะสมจากจุด 1 | เลือกจุดนอก Tree ที่มี T น้อยที่สุด

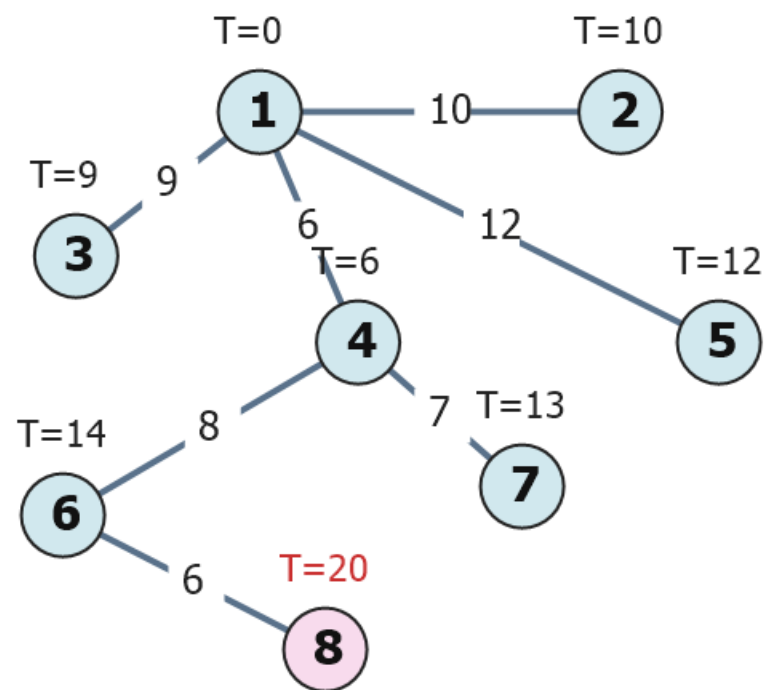
(1) เส้นทางที่พิจารณา: คำนวณ T



ค่า T ของจุดนอก Tree ที่ไปถึงได้:

8: 20 | 9: 21

(2) เพิ่มจุด 8: $T = 20$



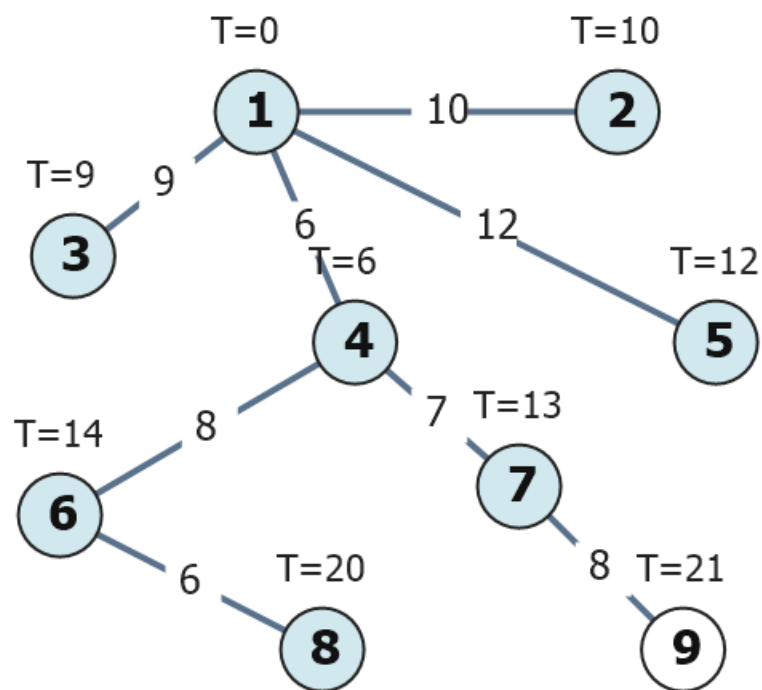
ผ่านเส้น 6-8: $14 + 6 = 20$

Tree = {1, 2, 3, 4, 5, 6, 7, 8}

กราฟที่ 2 | 6. Shortest Path (Dijkstra)

รอบที่ 8 | T = น้ำหนักสะสมจากจุด 1 | เลือกจุดนอก Tree ที่มี T น้อยที่สุด

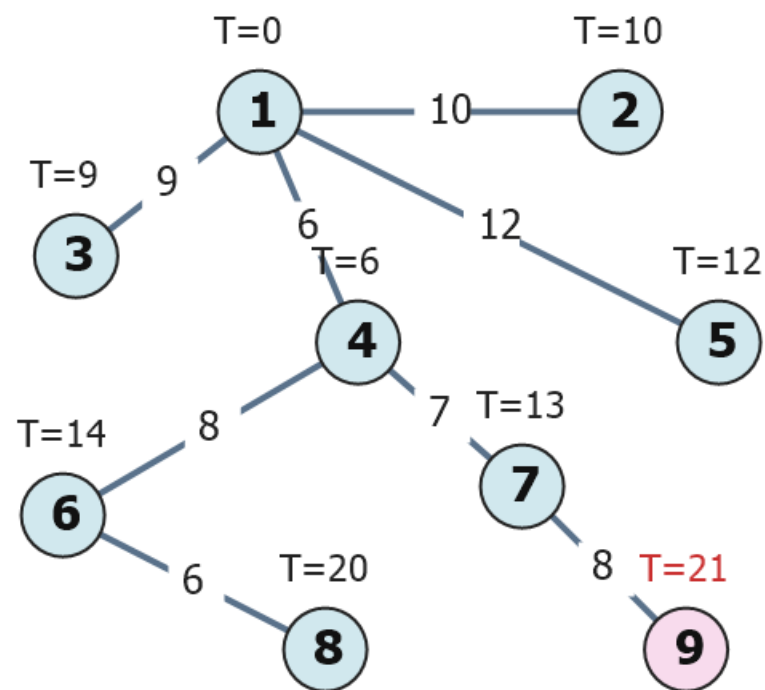
(1) เส้นทางที่พิจารณา: คำนวณ T



ค่า T ของจุดนอก Tree ที่ไปถึงได้:

9: 21

(2) เพิ่มจุด 9: $T = 21$



ผ่านเส้น 7-9: $13 + 8 = 21$

Tree = {1, 2, 3, 4, 5, 6, 7, 8, 9}

กราฟที่ 2 | Dijkstra: ตารางน้ำหนักสะสม

เริ่มด้วยจุด 1 ใน Tree | $T(1) = 0$ | ช่องตารางเป็นค่าหลังอัปเดตเพื่อนบ้านของจุดที่เพิ่ม

รอบ	เพิ่มจุด	T(1)	T(2)	T(3)	T(4)	T(5)	T(6)	T(7)	T(8)	T(9)
เริ่ม	1	0	10	9	6	12	∞	∞	∞	∞
1	4	0	10	9	6	12	14	13	∞	∞
2	3	0	10	9	6	12	14	13	∞	∞
3	2	0	10	9	6	12	14	13	∞	∞
4	5	0	10	9	6	12	14	13	∞	25
5	7	0	10	9	6	12	14	13	21	21
6	6	0	10	9	6	12	14	13	20	21
7	8	0	10	9	6	12	14	13	20	21
8	9	0	10	9	6	12	14	13	20	21

ค่าเท่ากันเลือกจุดเลขน้อยก่อน | เส้นทางใหม่เท่าค่าเดิมให้เก็บจุดก่อนหน้าเดิม

● กราฟที่ 2 | Dijkstra: เส้นทางสั้นที่สุดจากจุด 1

ผลลัพธ์มีทั้งเส้นทางและน้ำหนักสะสมตามโจทย์

ปลายทาง	เส้นทางจากจุด 1	น้ำหนักสะสม
1	1	0
2	1 → 2	10
3	1 → 3	9
4	1 → 4	6
5	1 → 5	12
6	1 → 4 → 6	14
7	1 → 4 → 7	13
8	1 → 4 → 6 → 8	20
9	1 → 4 → 7 → 9	21